

DATA PRIVACY **AND SECURITY**

Prof. Daniele Venturi

Chapter 1: Crypto 101

Master's Degree in Data Science
Sapienza University of Rome



Security and Cryptography

- Involves capabilities from **different areas**
 - Mathematics, physics, computer science, networking, law, and more
- How to build a **secure** system?
 - Many aspects to consider
 - **Cryptography**: The heart of any secure system
 - **Other aspects**: Physical security, logical security, security governance, security of code and implementations



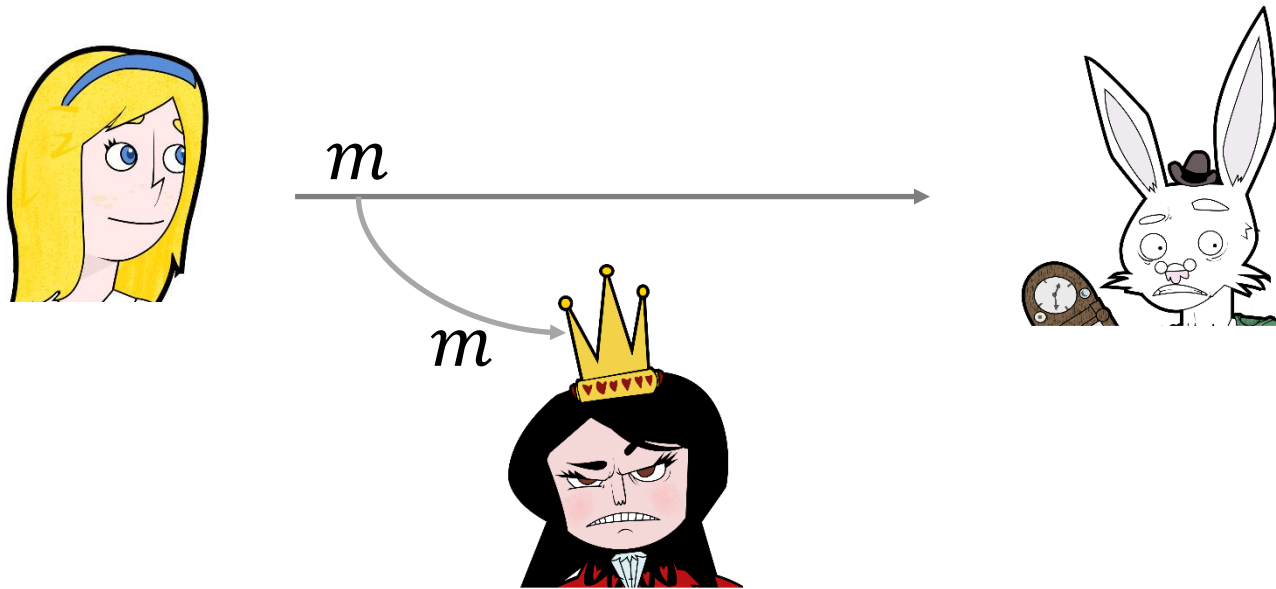
Provable Security

- In the past: The **ancient art** of secure communication
 - Examples: Caesar cipher, ENIGMA, one-time pad
- Today: A **real science**
 - Thanks to pioneers such as Silvio Micali, Shafi Goldwasser, Oded Goldreich
 - Formal **definitions** and security **proofs**
- We will give a very **high-level** overview
 - Focus on applications



Confidential Communication

- Alice wants to send a message to Bob over some communication channel
- Eve can **listen** to the channel
- How to protect the **message content**?

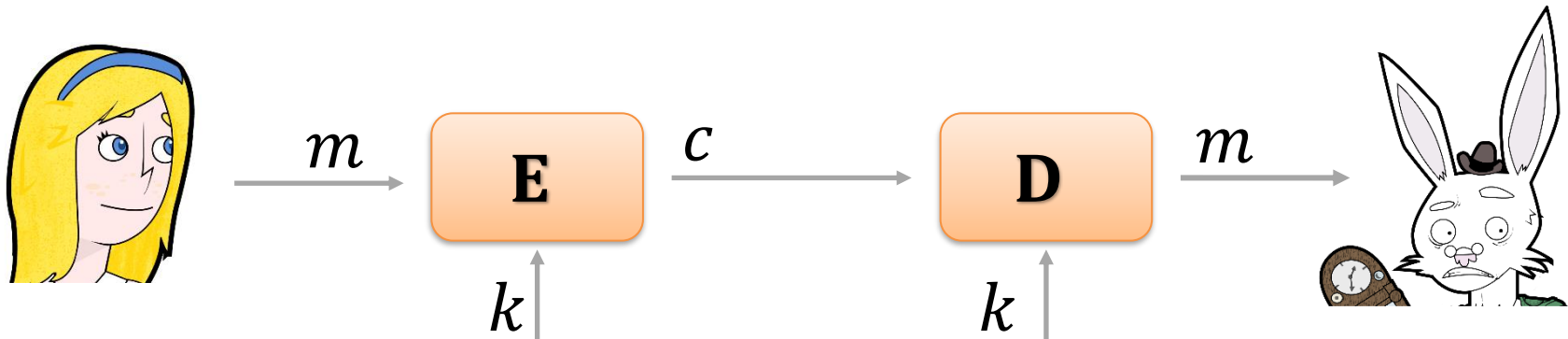


Secret-key Cryptography



Secret-Key Encryption

- Assume Alice and Bob **share** a secret key



- Correctness:** $\mathbf{D}(k, \mathbf{E}(k, m)) = m$
- Kerckhoff's principle:** Security only based on the secrecy of the key (**algorithms are public**)



Perfect Secrecy

- Definition due to Claude Shannon (1949)
 - Ciphertext **reveals nothing** about the plaintext

$$\Pr[M = m] = \Pr[M = m | C = c]$$

- One-time pad (binary version):
 - $\mathbf{E}(k, m) = k \oplus m$
 - $\mathbf{D}(k, c) = k \oplus c$
- Limitations:
 - **One key** per message, and message **as long as key**
 - Can be shown to be **inherent**



Computational Security

- Previous definition is information-theoretic
 - Holds even for **all-powerful** adversaries
 - Unconditional security, i.e., **no assumptions!**
- Natural relaxation: **Computational security**
 - Computationally bounded adversary (PPT Turing machine)
 - Adversary has negligible probability of success (e.g., 2^{-80})
- Advantage: **Single short key** for encrypting an **unbounded** number of messages



AES (Rijndael)

- A widely used **blockcipher**
 - Created to replace DES
- NIST call for proposals in 1997
 - Evaluation criteria: Security, costs, intellectual property, implementation and execution, versatility, key agility, simplicity
- Two rounds were performed; 15 algorithms were selected in the first and 5 in the second
 - NIST completed the evaluation on October 2, 2000, and selected **Rijndael (Daemen + Rijmen)**



AES Structure

- Block length of 128 bits (16 bytes)
 - Three key sizes: 128, 192, or 256 bits
- # of rounds: 10, 12, or 14
 - Let $s_{i,j}^{(\text{in})}$ be 1 byte of the **state** at a given round (initially the first plaintext block)
 - The secret key is used to compute the **sub-keys** $k_{i,j}^{(r)}$, one for each round r
 - In each round state subject to 4 operations: **SubBytes**, **ShiftRows**, **MixColumns**, **AddRoundKey**



Arithmetic in $GF(2^8)$

- AES uses the Galois Field $GF(2^8)$
 - 1 byte $\Rightarrow$ 8 bits $\Rightarrow$ 2 hexadecimal digits
 - Example: $[01101100]_2 = [6C]_{16}$
- Interpret each byte as the binary coefficients of a degree-7 polynomial
 - Sum of 2 polynomials is still a degree-7 polynomial
 - Multiplication might increase the degree
 - Modular reduction w.r.t. **irreducible polynomial**

$$h(X) = X^8 + X^4 + X^3 + X + 1$$

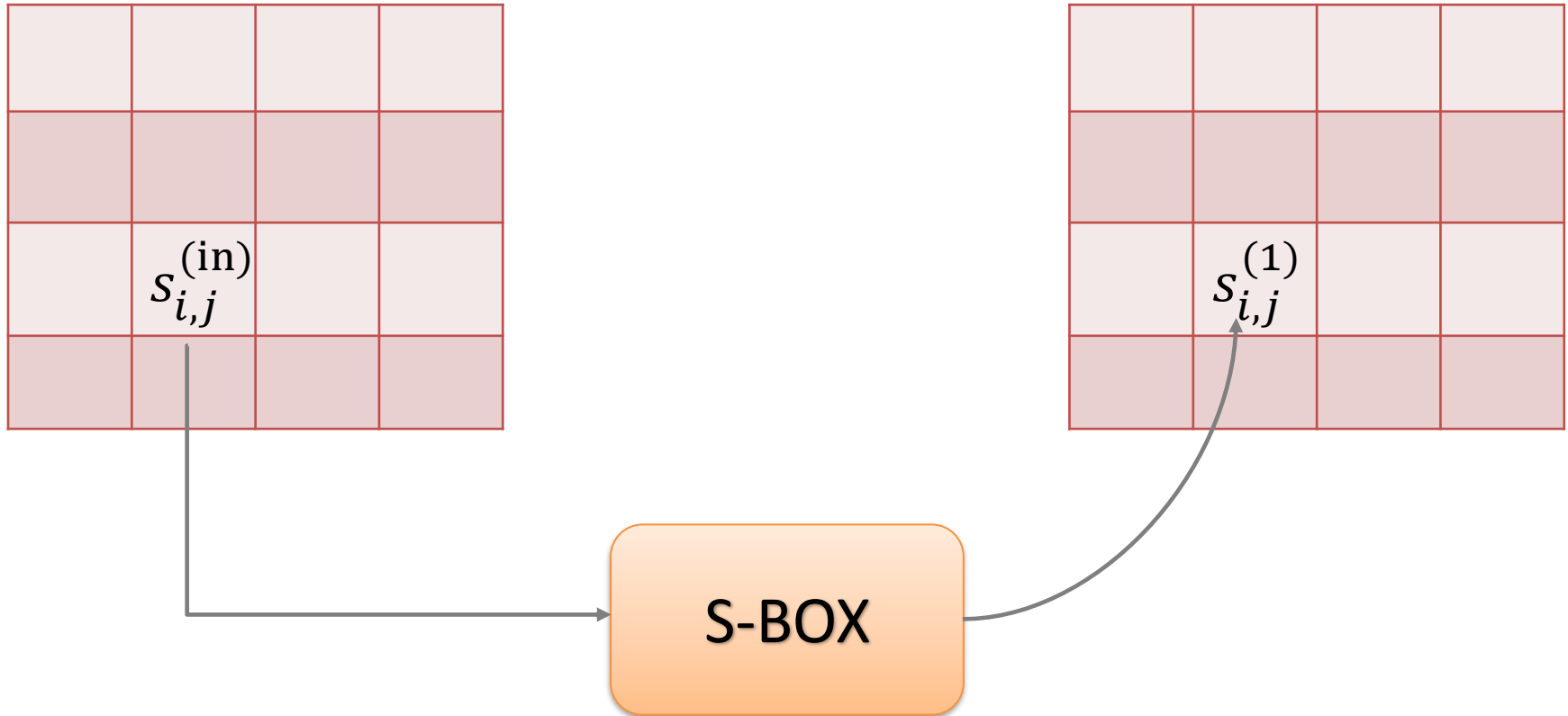


Arithmetic in $GF(2^8)$: Example

- $[53]_{16} \cdot [CA]_{16} = [01]_{16}$ in $GF(2^8)$
 - $[53]_{16} = X^6 + X^4 + X + 1$
 - $[CA]_{16} = X^7 + X^6 + X^3 + X$
- $[53]_{16} \cdot [CA]_{16} = X^{13} + X^{12} + X^{11} + X^{10} + X^9 + X^8 + X^7 + X^6 + X^5 + X^4 + X^3 + X^2 + X + 1$
 - By performing **long division**, it is easy to check that $X^{13} + X^{12} + X^{11} + X^{10} + X^9 + X^8 + X^7 + X^6 + X^5 + X^4 + X^3 + X^2 + X + 1$ gives $X^5 + X^4 + X^3 + X + 1$ with **remainder** 1



SubBytes



AES S-BOX

- A simple **substitution box** (lookup table)
- It maps 8-bit inputs to 8-bit outputs
 - Let $x = [x_7, x_6, x_5, x_4, x_3, x_2, x_1, x_0]_2$ be the input
 - Map x into its **multiplicative inverse** z modulo $h(X)$ and apply an **affine transformation**:

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \\ y_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} z_0 \\ z_1 \\ z_2 \\ z_3 \\ z_4 \\ z_5 \\ z_6 \\ z_7 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$



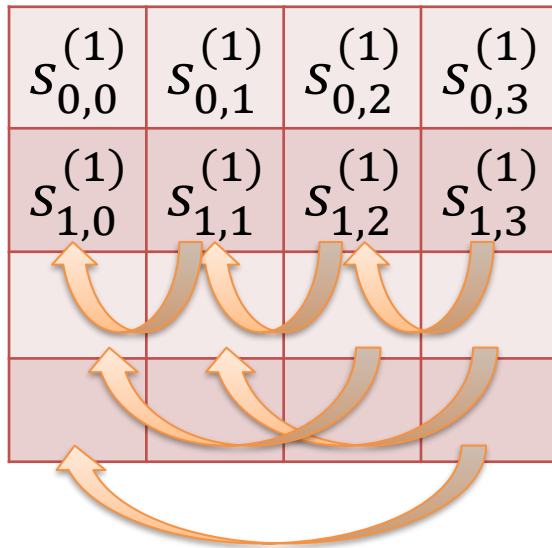
AES S-BOX

- Input: [68]
- Output: [45]

		y															
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
x	0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
	1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
	2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
	3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
	4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
	5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
	6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
	7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
	8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
	9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
	a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
	b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
	c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
	d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
	e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
	f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16



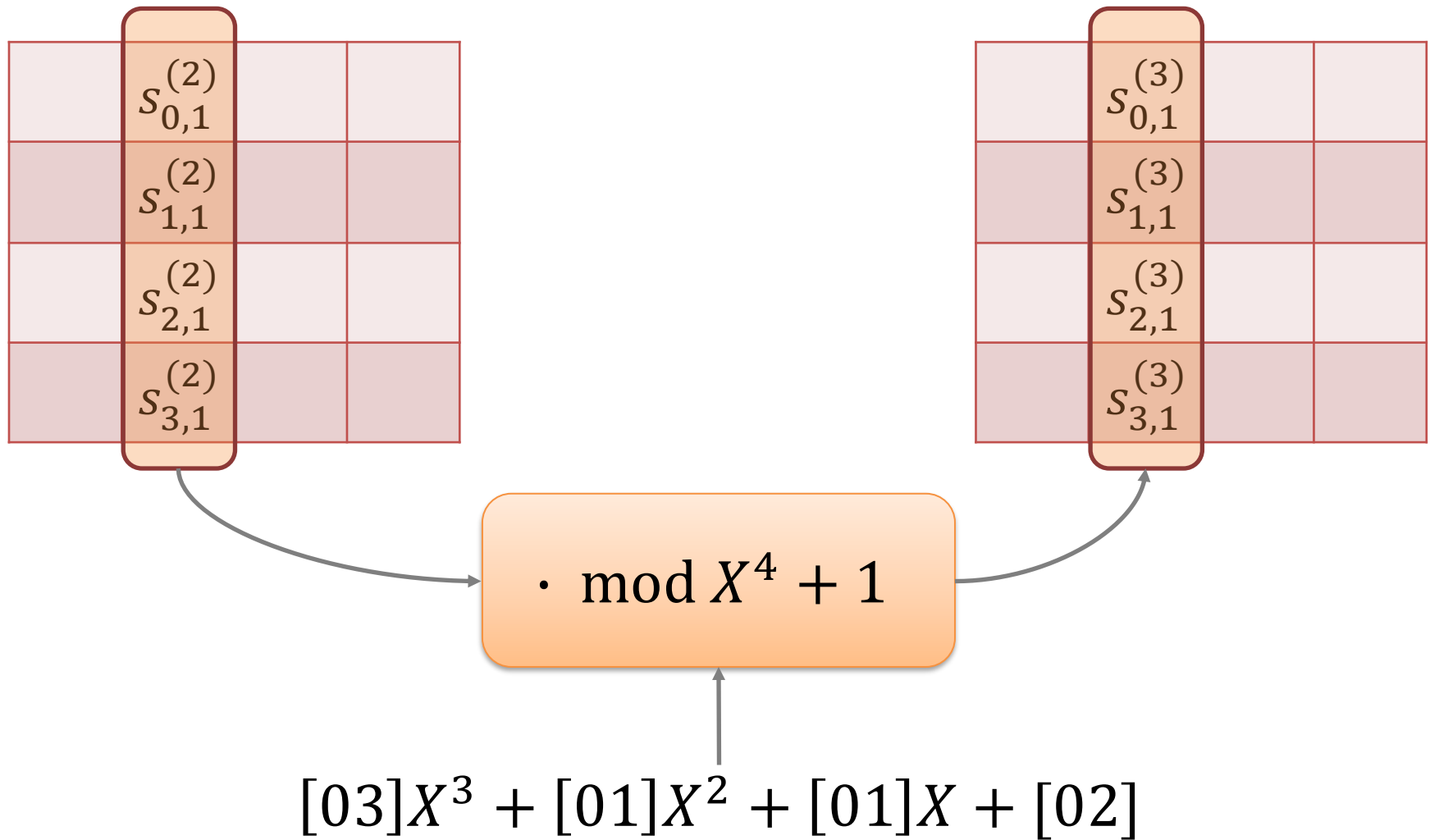
ShiftRows



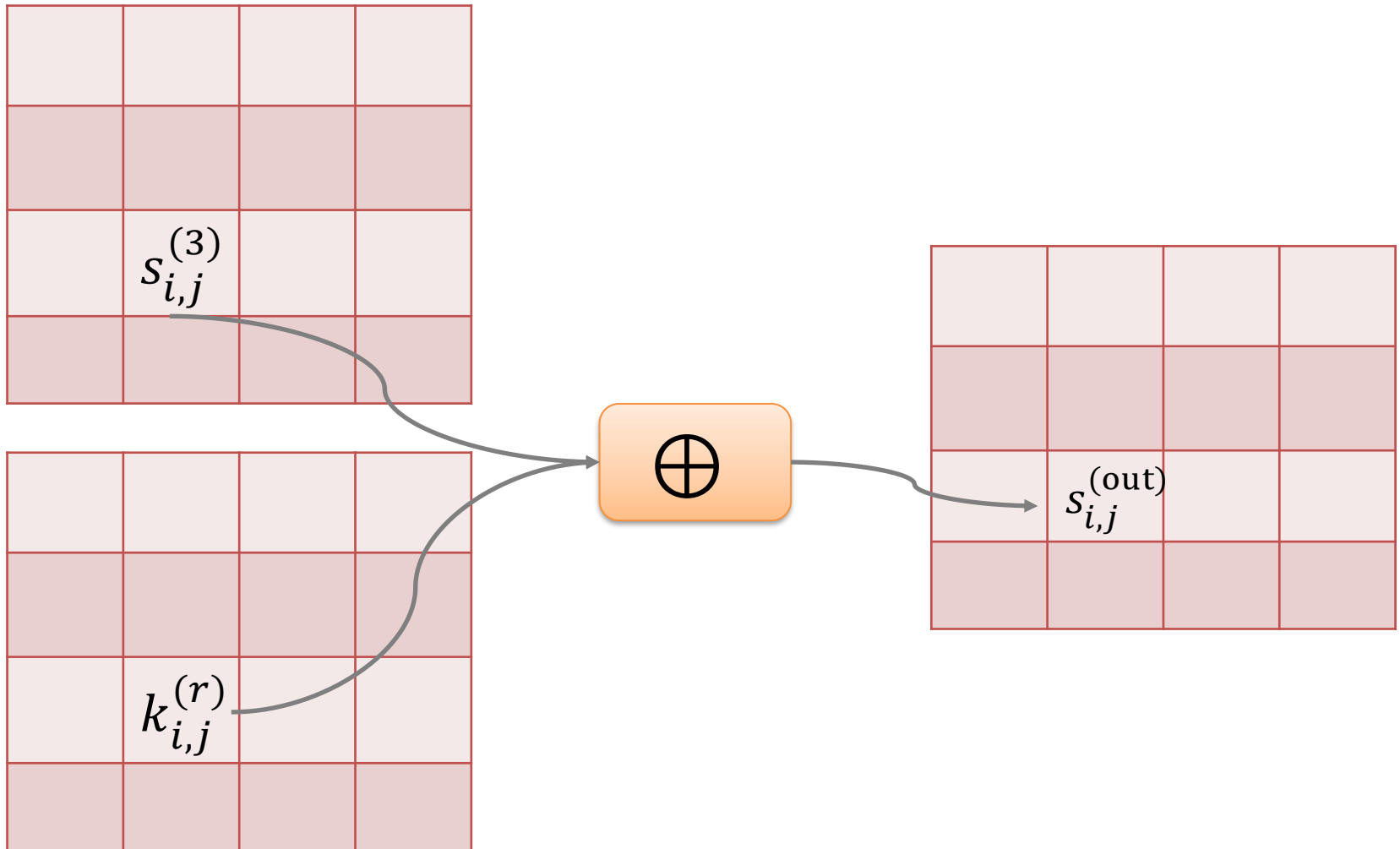
$$\begin{bmatrix} s_{0,j}^{(2)} \\ s_{1,j}^{(2)} \\ s_{2,j}^{(2)} \\ s_{3,j}^{(2)} \end{bmatrix} = \begin{bmatrix} s_{0,j}^{(1)} \\ s_{1,j-1}^{(1)} \\ s_{2,j-2}^{(1)} \\ s_{3,j-3}^{(1)} \end{bmatrix}$$



MixColumns



AddRoundKey

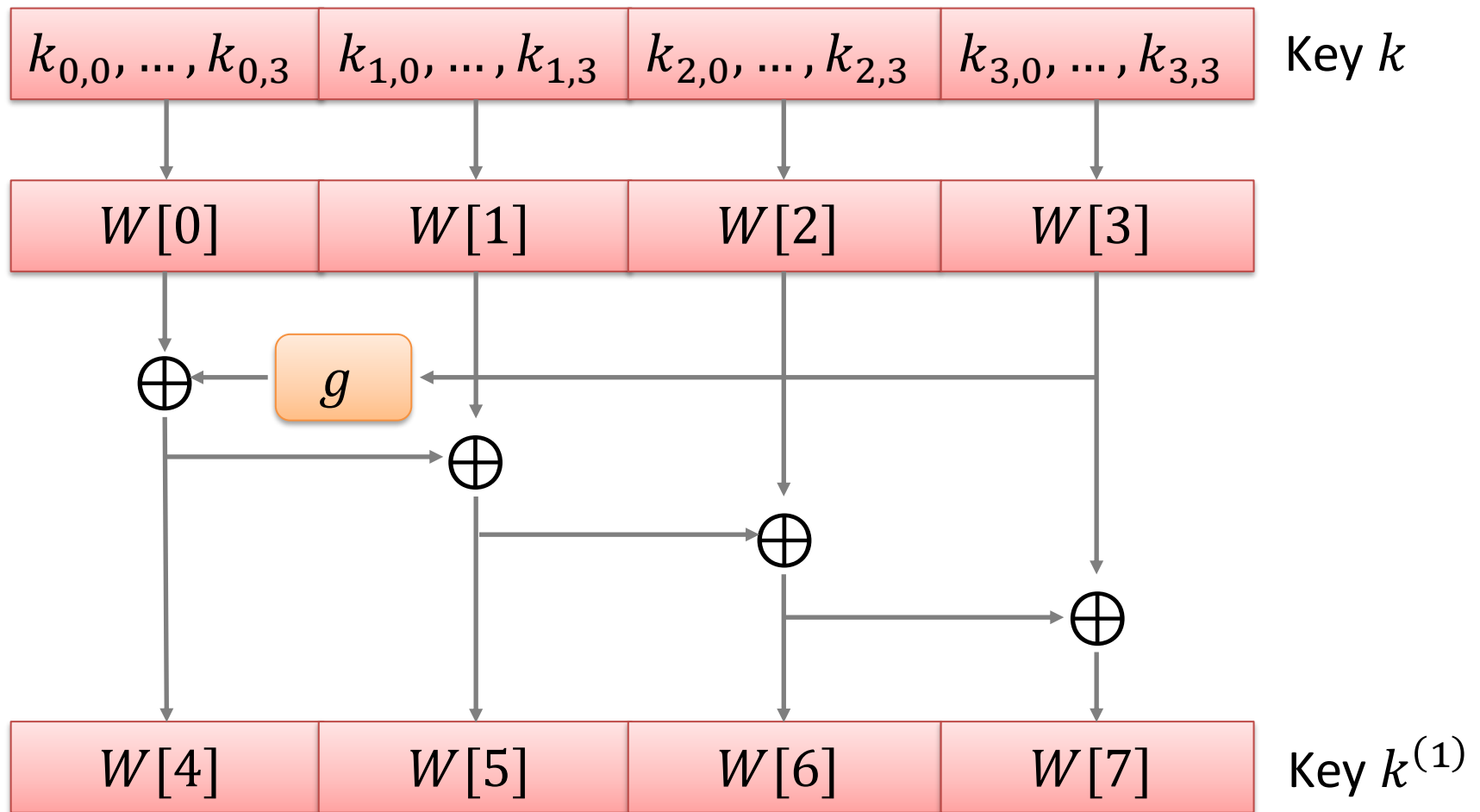


Key Schedule

- Takes the original key k (128, 192, or 256 bits) and **derives sub-keys** $k^{(r)}$, for $r = 10, 12, 14$
- When $|k| = 128$ and $r = 10$:
 - Key expansion array W with 44 32-bit elements
 - $W[0], \dots, W[3]$ equal to the original key (used for initial XOR with the plaintext – key whitening)
 - $W[4i] = W[4(i - 1)] + g(W[4i - 1])$
 - $W[4i + j] = W[4i + j - 1] + W[4(i - 1) + j]$
 - g is a non-linear function (based on the S-Box)



Key Schedule



Security of AES

- No **practical attack** is known
 - Best attacks break AES with 7 rounds (128-bit key), 8 rounds (192-bit key) and 9 rounds (256-bit key)
- Brute force is out of reach: $3,4 \cdot 10^{38}$ possible combinations (128-bit key)
 - Best brute force attack took 5 years to crack a 64-bit key using thousands of CPUs
- But AES comes with **no proof of security!**

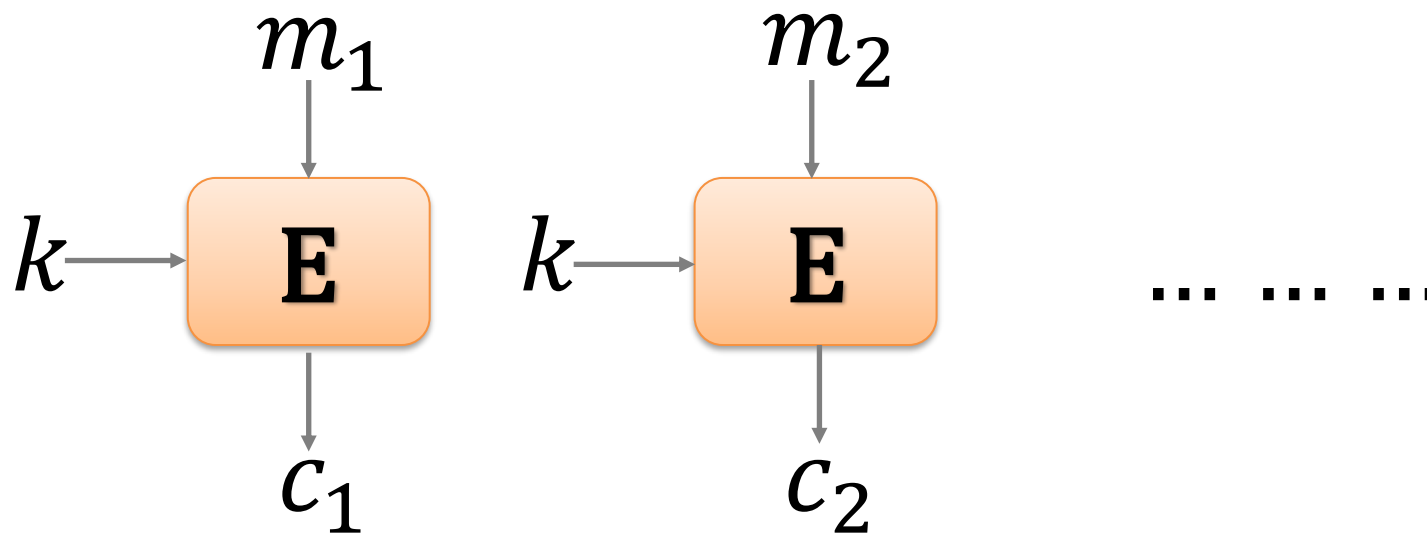


Modes of Operation

- Block ciphers encrypt **fixed size blocks**
 - E.g., AES block size is 128 bits
- Plaintext messages may have an **arbitrary length**:
Use a **mode of operation**
 - Segment data & encrypt and chain multiple blocks
 - Might require to pad messages to make their length a multiple of the block size
- 4 modes defined for DES in ANSI standard "ANSI X3.106-1983 Modes of Use"



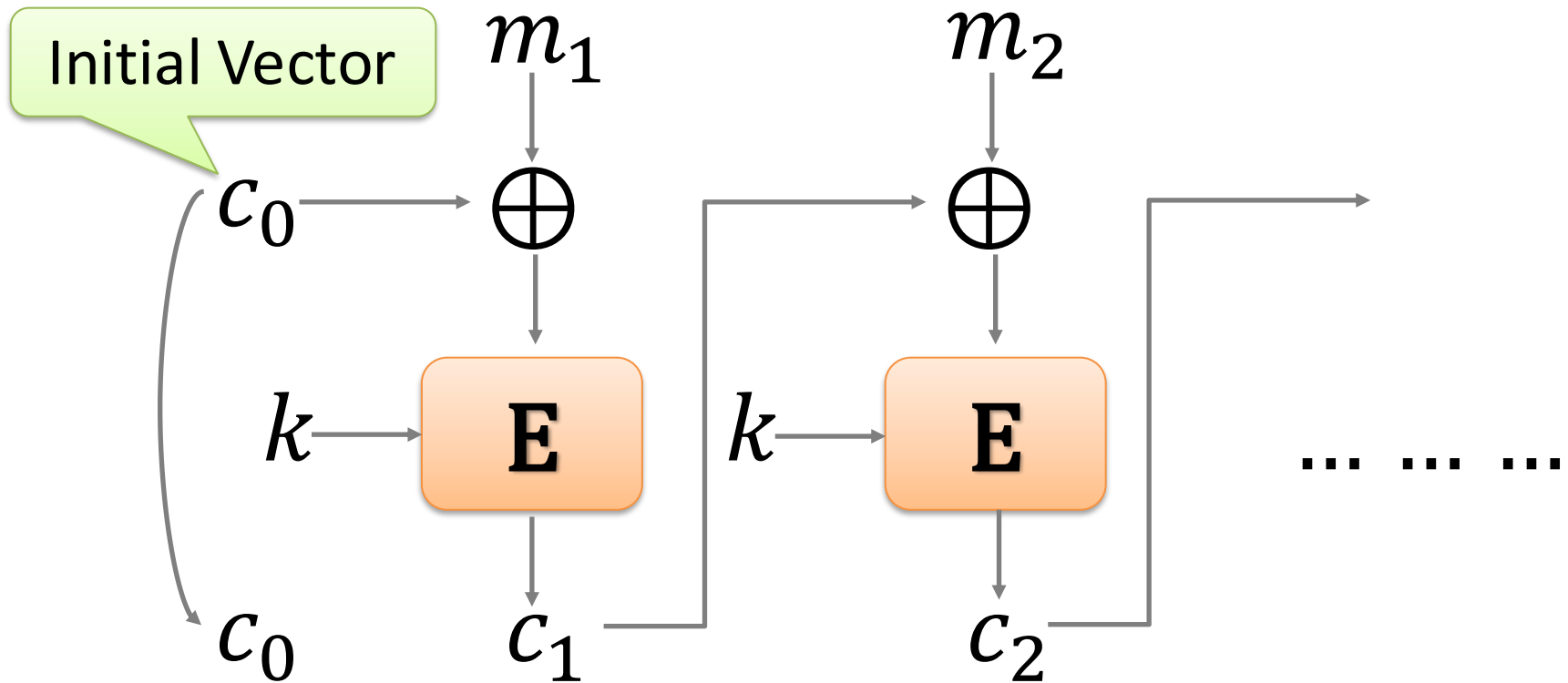
ECB Mode



$$\forall i: c_i = \mathbf{E}(k, m_i)$$



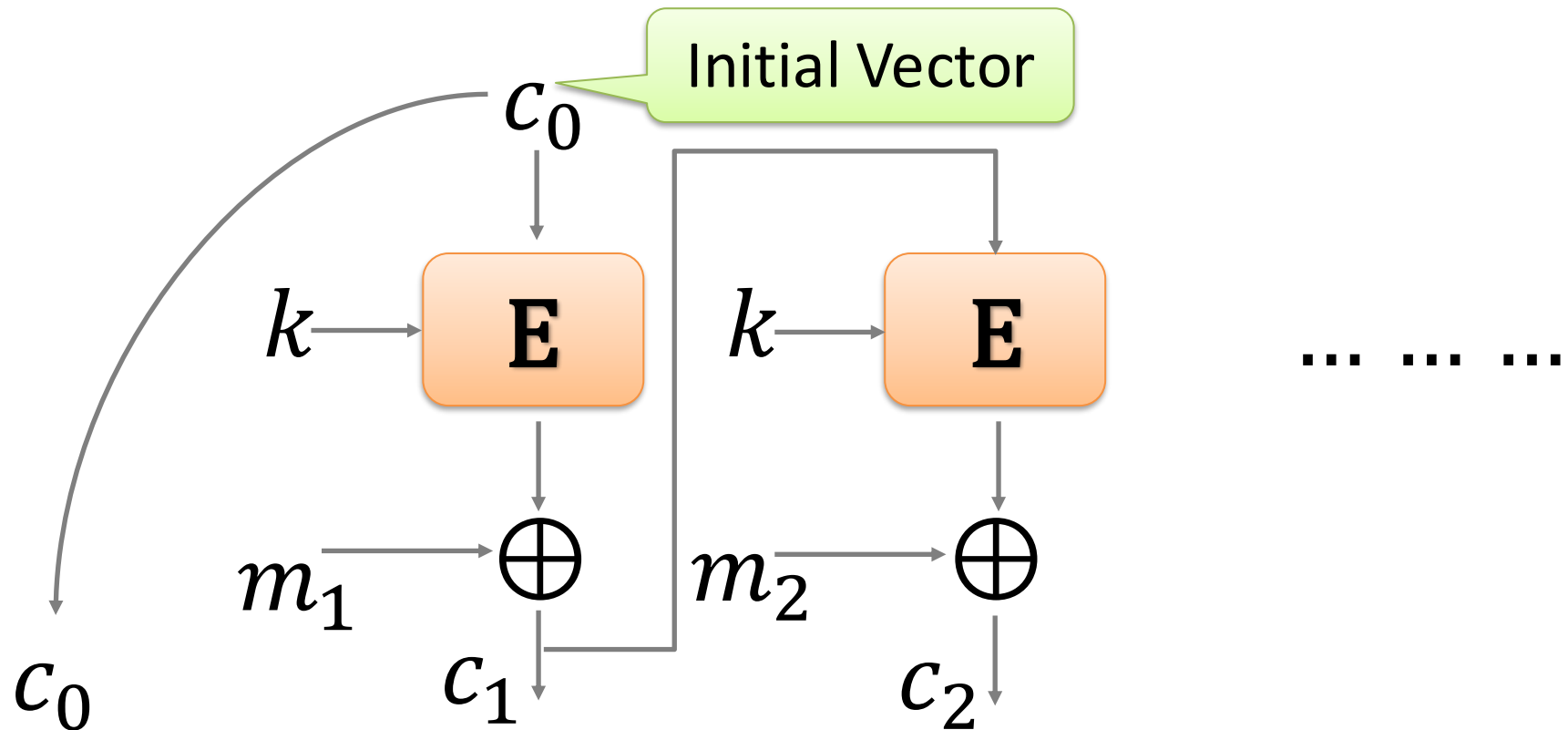
CBC Mode



$$\forall i: c_i = \mathbf{E}(k, c_{i-1} \oplus m_i)$$



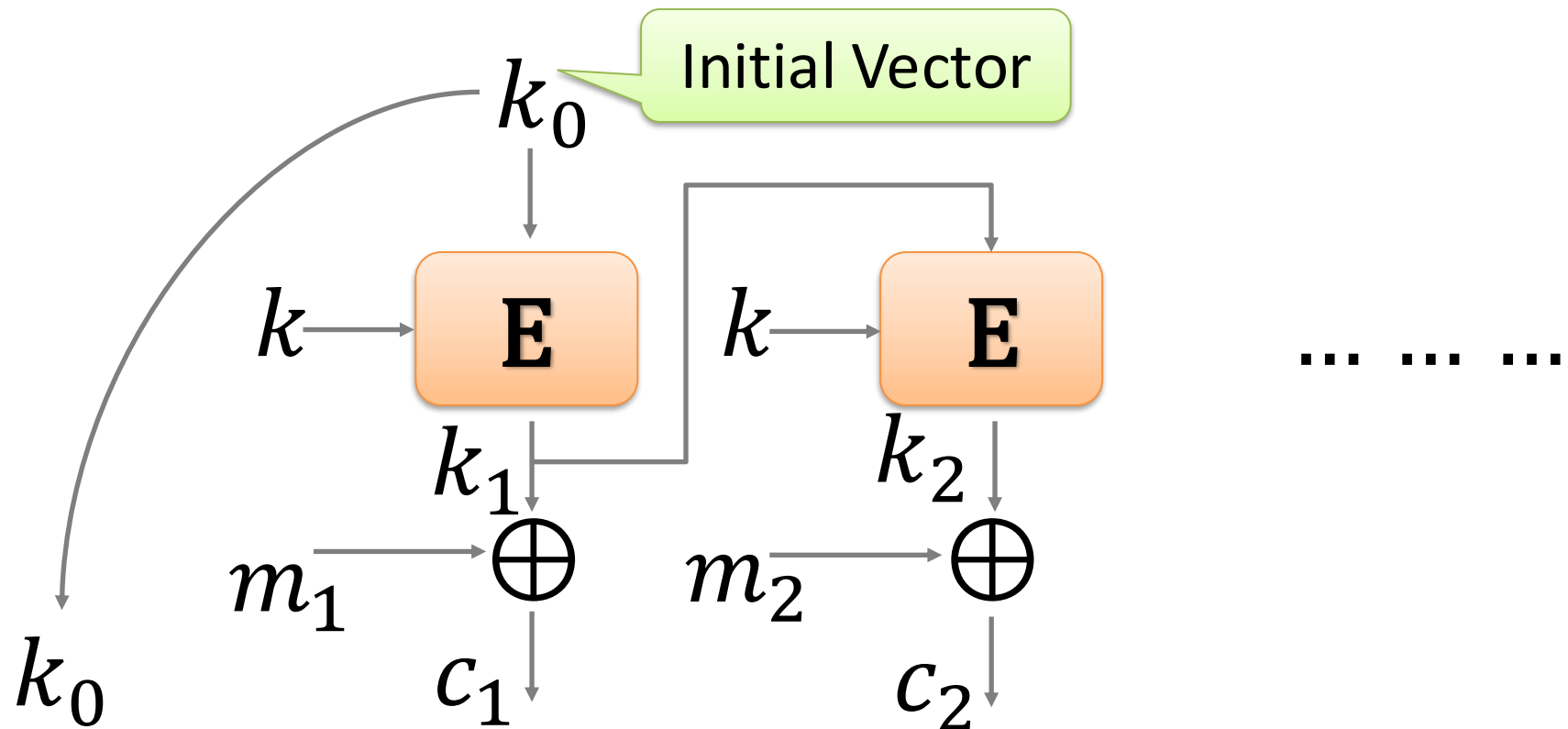
CFB Mode



$$\forall i: c_i = m_i \oplus \mathbf{E}(k, c_{i-1})$$



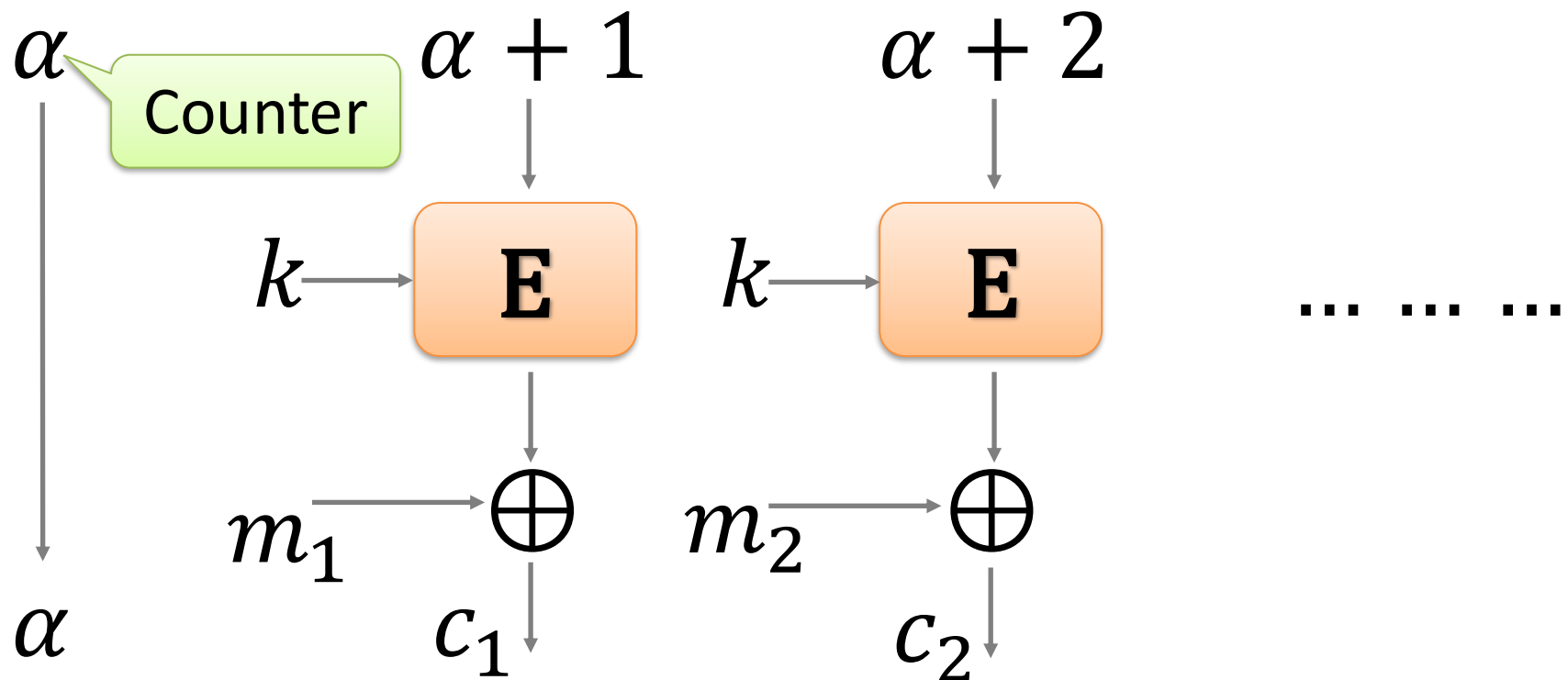
OFB Mode



$$\forall i: c_i = m_i \oplus \mathbf{E}(k, k_{i-1})$$



CTR Mode



$$\forall i: c_i = m_i \oplus \mathbf{E}(k, (\alpha + i) \bmod 2^n)$$



Comparison

- ECB: Identical plaintext blocks are encrypted into **identical** ciphertext blocks

Mode	Enc	Dec	\$Access	Security
ECB	✓	✓	✓	X
CBC	X	✓	✓	✓
CFB	X	✓	✓	✓
OFB	X	X	X	✓
CTR	✓	✓	✓	✓



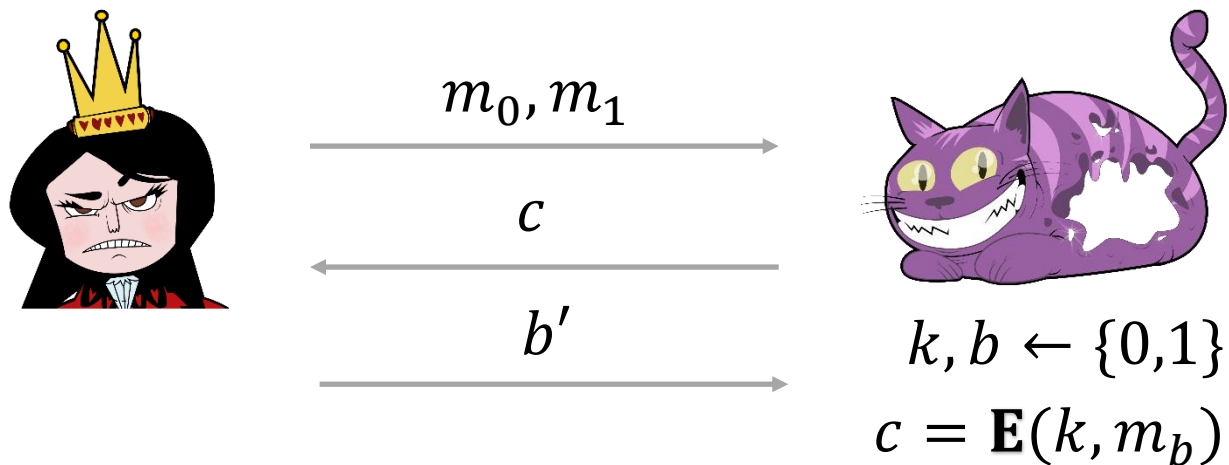
Security of Block Ciphers

- Rule of thumb (Shannon): A good block cipher should have both **confusion** and **diffusion**
 - Confusion means there is a **complex relation** between ciphertext and plaintext
 - Diffusion roughly means that a **one-bit** flip in the plaintext changes **each bit** of the ciphertext with probability $\approx 1/2$
- But can we define more **formally** what it means for a cipher to be secure?



One-Time Security

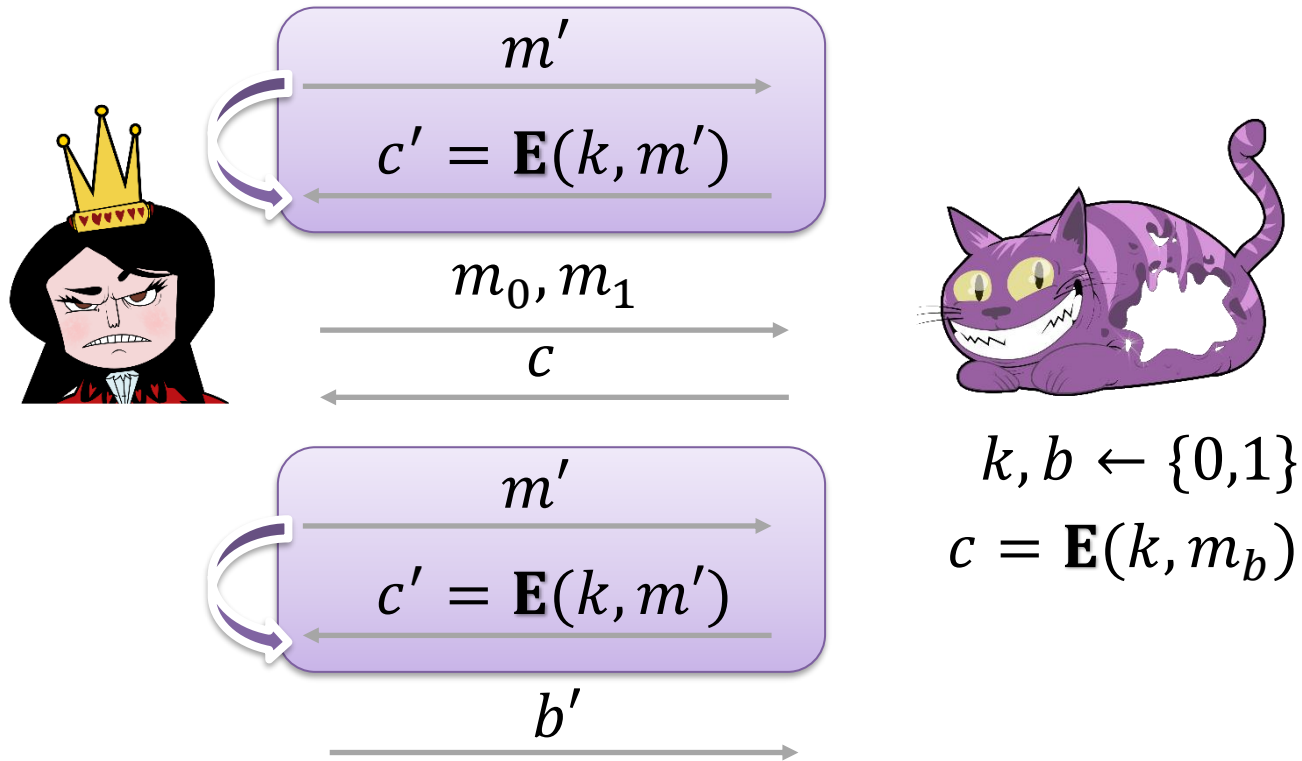
- The **indistinguishability** paradigm



- Hard to **guess** b w.p. better than $1/2$
- No encryption/decryption capabilities



Chosen-Plaintext Attacks (CPA) Security

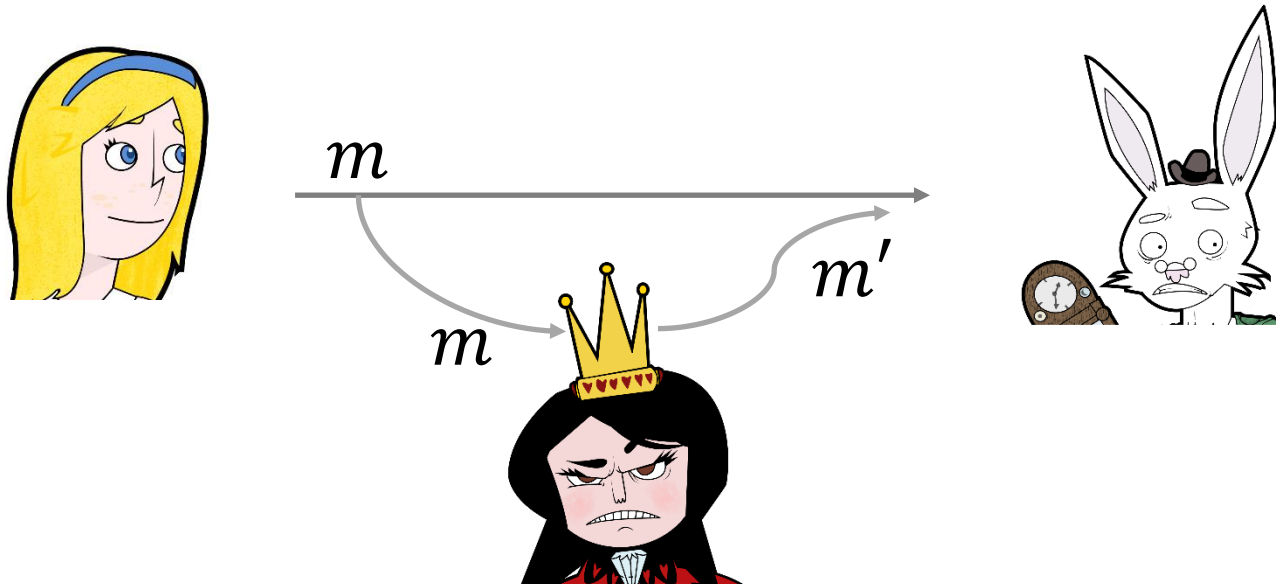


- Adversary can ask **encryption** queries
- Requires **randomness**!



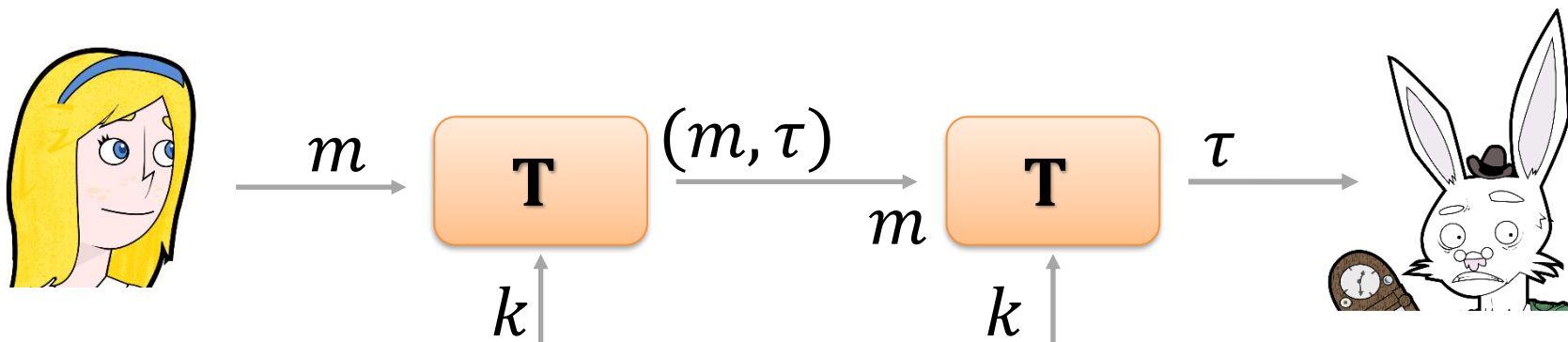
Authenticated Communication

- Alice wants to send a message to Bob over some communication channel
- Eve can **modify** the message
- How to protect the message **authenticity**?



Message Authentication Codes

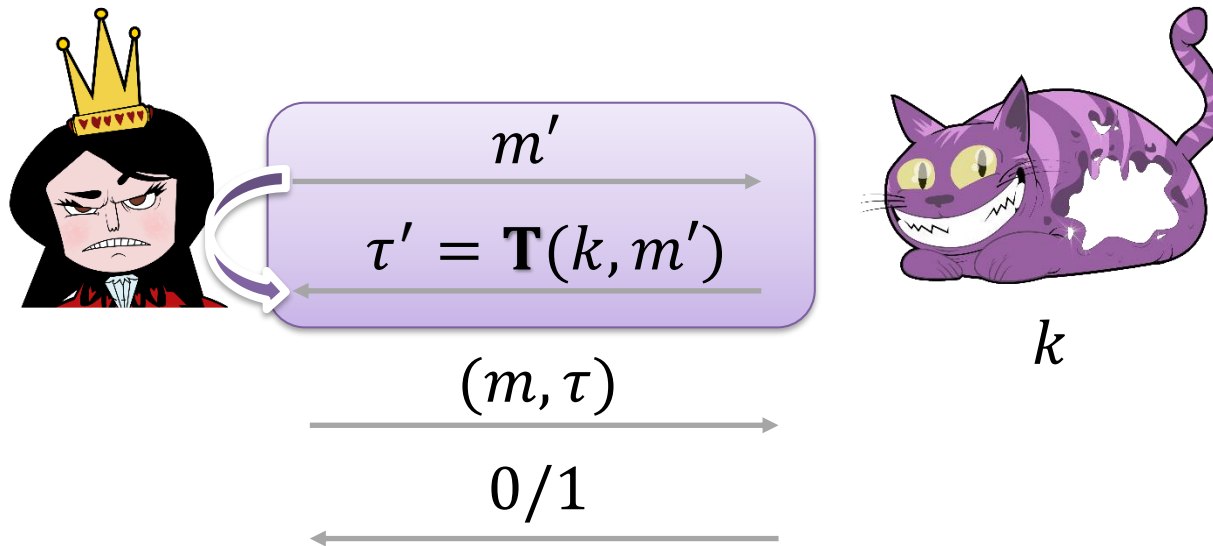
- Assume Alice and Bob **share** a secret key



- Correctness:** By definition
- Security:** Should be hard to **forge** a tag on a message without knowing the key



Unforgeability



- Adversary wins iff (m, τ) is **valid** and m is **fresh** (i.e. not asked during tag queries)
 - Reply attacks not covered by definition



CBC-MAC

- Use AES in CBC mode
- Fix $IV = 0^n$ and output **only** the last block
 - I.e., for $m = (m_1, \dots, m_t)$ where $m_i \in \{0,1\}^n$ compute $\tau_i = \mathbf{F}(k, \tau_{i-1} \oplus m_i)$, where $\tau_0 = IV$, and return $\tau = \tau_t$
 - Only secure for **fixed length** messages, for variable length messages need to encrypt the output with an independent key (i.e., $\tau' = \mathbf{F}(k', \tau)$)
 - Insecure in case **all blocks** are output



Why Fixed Length?

- Suppose we use CBC-MAC to authenticate **variable length** messages
- Adversary picks arbitrary $m_1, m_2 \in \{0,1\}^n$ and obtains tags on m_1 and $m_2 \oplus \tau_1$

$$\tau_1 = \mathbf{F}(k, m_1); \tau_2 = \mathbf{F}(k, m_2 \oplus \tau_1)$$

- Output forgery $m^* = m_1 || m_2$ and $\tau^* = \tau_2$

$$\tau_2 = \mathbf{F}(k, m_2 \oplus \tau_1) = \mathbf{F}(k, \mathbf{F}(k, m_1) \oplus m_2))$$



Why Only the Last Block?

- Suppose CBC-MAC outputs **all blocks**
- Adversary picks arbitrary $m_1, m_2 \in \{0,1\}^n$ and obtains tag $\tau_1 || \tau_2$ on $m_1 || m_2$

$$\tau_1 = \mathbf{F}(k, m_1); \tau_2 = \mathbf{F}(k, m_2 \oplus \tau_1)$$

- Output forgery $m^* = \tau_1 \oplus m_2 || \tau_2 \oplus m_1$ and $\tau^* = \tau_2 || \tau_1$

$$\mathbf{F}(k, \mathbf{F}(k, m_2 \oplus \tau_1) \oplus \tau_2 \oplus m_1) = \mathbf{F}(k, m_1)$$

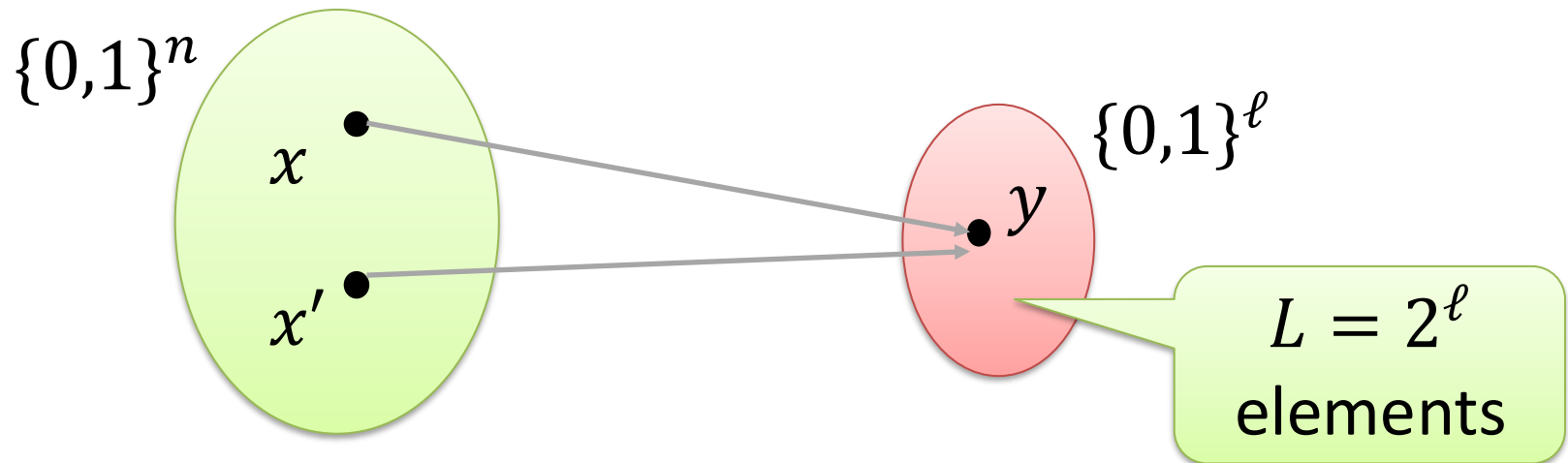


Why not a Random IV?

- Suppose that for each tag we sample **random** $\tau_0 \leftarrow_{\$} \{0,1\}^n$ and output (τ_0, τ_t) as tag
 - Here, t is the number of n -bit blocks in a message
- Adversary picks arbitrary $m \in \{0,1\}^n$ and obtains tag (τ_0, τ_1) , where $\tau_1 = \mathbf{F}(k, \tau_0 \oplus m)$
- Output forgery $m^* = \tau_0$ and $\tau^* = (m, \tau_1)$



Cryptographic Hashing



- Security properties:
 - **One wayness**: Given y , find x such that $\mathbf{H}(x) = y$
 - **Weak collision resistance**: Given x , find $x' \neq x$ s.t. $\mathbf{H}(x) = \mathbf{H}(x')$
 - **Strong collision resistance**: Find x and x' s.t. $\mathbf{H}(x) = \mathbf{H}(x')$ but $x \neq x'$



Brute Force Attacks

- Assume **H** to be a **random hash function**
- **One wayness**: Given y choose $x_1, \dots, x_q$ and hope that $\mathbf{H}(x_i) = y$ for some $i \in [q]$
 - Success probability: $\leq q/L$ (union bound)
- **Weak collision resistance**: Similar to above
- **Strong collision resistance**: Choose distinct $x_1, \dots, x_q$ and hope to find a collision

$$\Pr[\exists i \neq j: y_i = y_j] \leq \sum_{i \neq j} \Pr[y_i = y_j] \leq \frac{q^2}{2L}$$



The Birthday Paradox

- Suppose $y_1, \dots, y_q$ are **random**
 - Let $NoColl_i$ be the event that **no collision** occurs within $y_1, \dots, y_i$
 - $\Pr[NoColl_{i+1} | NoColl_i] = (1 - i/L)$

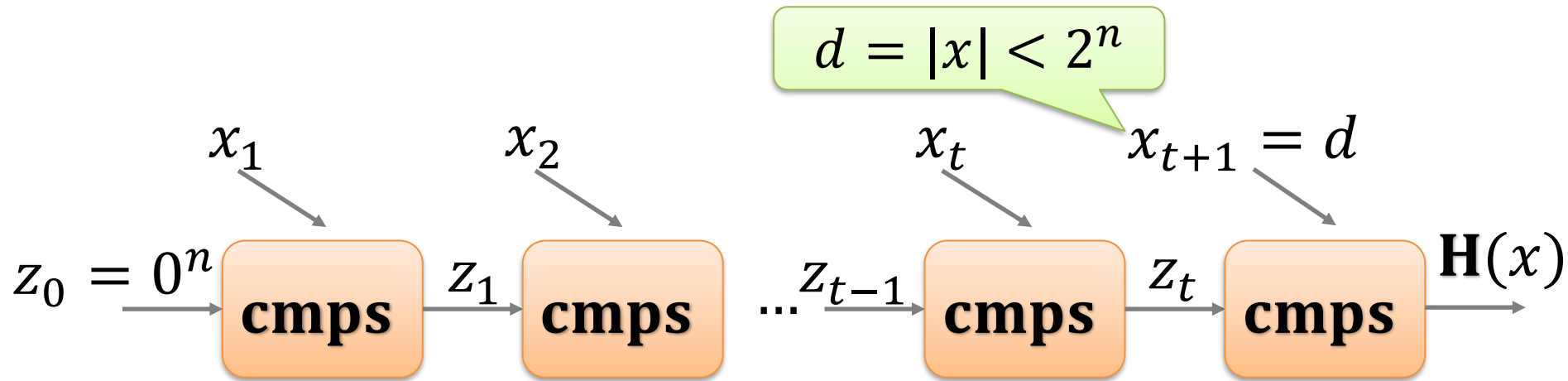
$$\begin{aligned}\Pr[NoColl_q] &= \prod_{i=1}^{q-1} \left(1 - \frac{i}{L}\right) \leq \prod_{i=1}^{q-1} e^{-\frac{i}{L}} = e^{-\sum_{i=1}^{q-1} \frac{i}{L}} \\ &= e^{-q(q-1)/2L}\end{aligned}$$

- Thus, $1 - \Pr[NoColl_q] \geq \frac{q(q-1)}{4L}$
 - Success w.p. $\geq 1/2$ whenever $q \approx \sqrt{L}$



Merkle-Damgaard

- Let **cmps** be a **compression function** outputting ℓ' bits out of ℓ bits
 - **cmps** is collision resistant, but domain is **fixed**
- A construction due to Merkle and Damgaard yields a collision resistant hash function for **arbitrary domains**



Davies-Meyer

- Compression functions can be constructed from **block ciphers**

$$\mathbf{cmps}(x_1, x_2) = x_2 \oplus \mathbf{AES}(x_1, x_2)$$

- Analysis requires to assume idealization of AES
 - Because the input is used **as the key**
 - Ideal cipher: Block-cipher as a **random permutation** for **every choice** of the key



Hash & MAC

- Typical (but **flawed**) construction of a MAC based on a hash function

$$\mathbf{T}(k, m) = \mathbf{H}(k || m)$$

- Attack based on **length extension** (for Merkle-Damgaard-based constructions)
 - Let $m^* = m || d || m_{t+1}$ and tag $\tau^* = \mathbf{cmps}(\mathbf{cmps}(\tau || m_{t+1}) || d$ - 2) for $\tau = \mathbf{H}(k || m)$ and $d = |m|$



HMAC

- Solution: Hash **twice**!

$$\mathbf{HMAC}(k, m) = \mathbf{H}(k^+ \oplus opad || \mathbf{H}(k^+ \oplus ipad || m))$$

- k^+ : Key k padded with zeroes to the left
- $opad$: 5C5C ... 5C (in HEX)
- $ipad$: 3636 ... 36 (in HEX)
- Internet **standard** RFC 2104
- Can work with any of SHA-2 or SHA-3

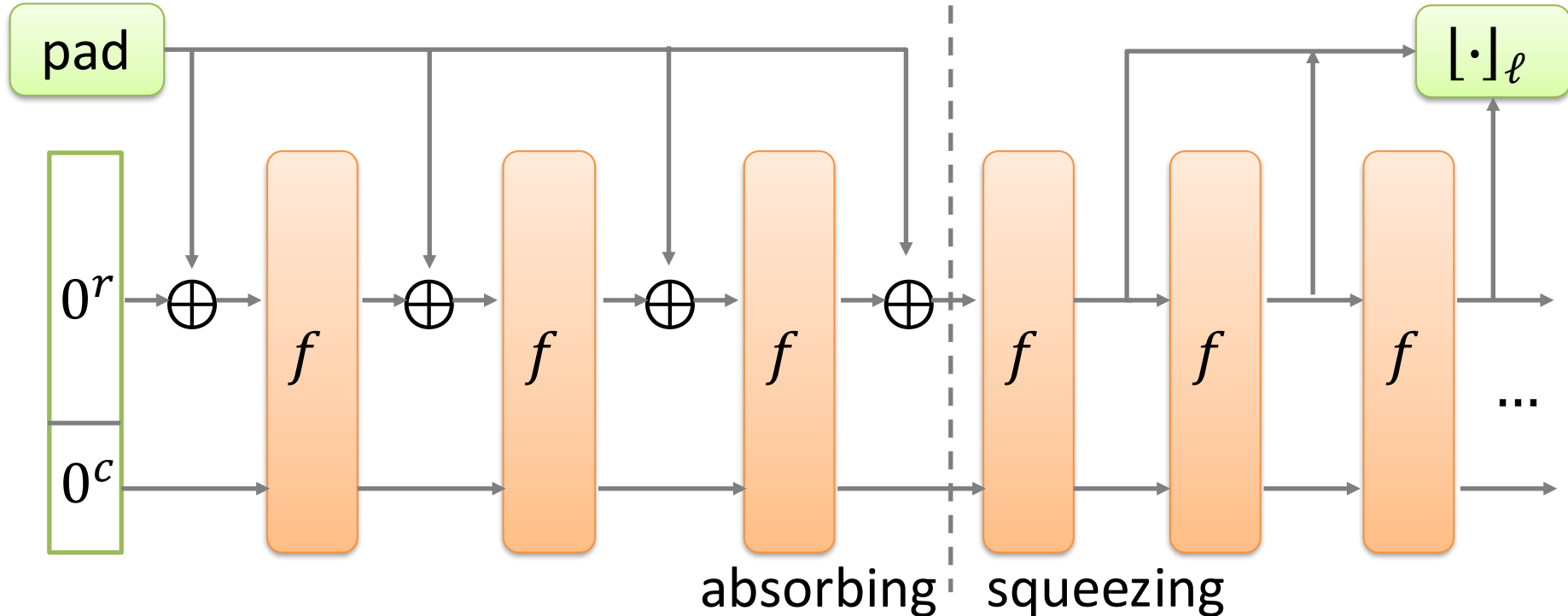


SHA-3

- 2005-2006: NIST thinks about SHA-3 contest
 - MD5 and SHA-1 were damaged by attacks
 - SHA-2 based on the same principles
- October 2008: Deadline for proposals
 - More efficient than SHA-2
 - Output lengths: 224, 256, 384, 512 bits
 - Security: collision resistant (weak and strong)
- October 2, 2012: NIST announces **Keccak** as SHA-3 winner



The Sponge Construction



- Can be used as a stream cipher, or a MAC too
- Security for **ideal** f is roughly $q(q - 1)/2^{c+1}$
 - $q = \#$ of calls to f

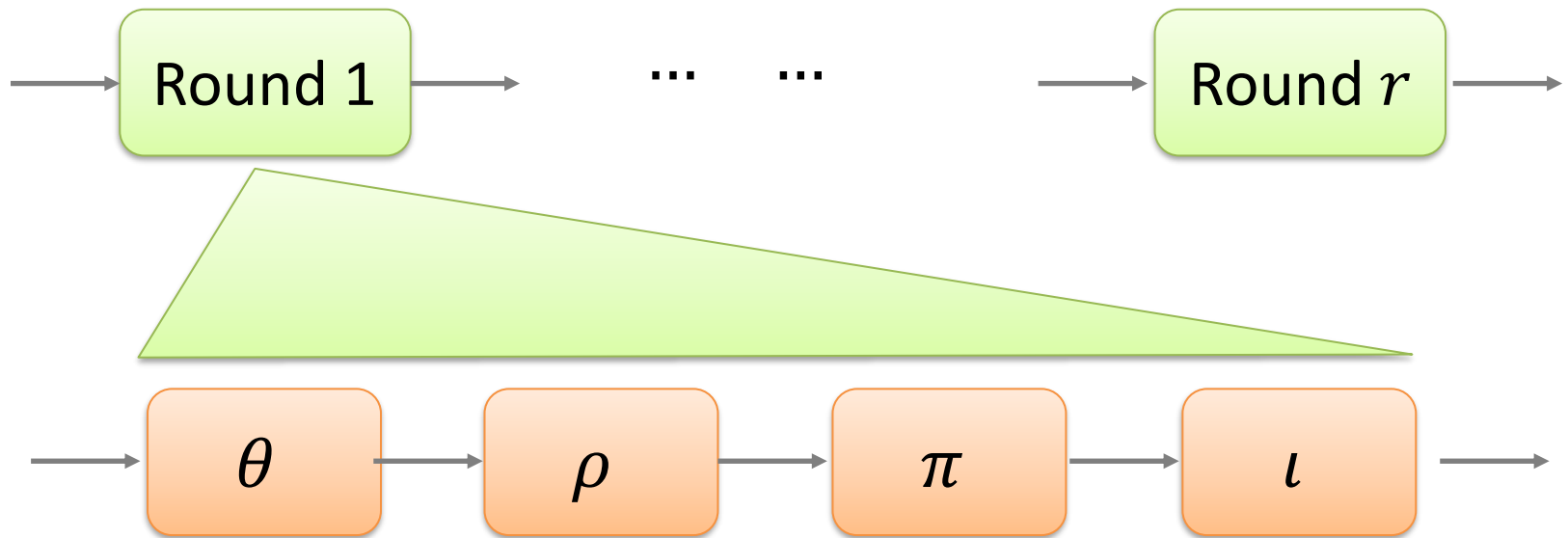


Inside Keccak

- **Absorbing**: The message blocks are padded and processed
- **Squeezing**: An output of configurable length is produced
- Parameters:
 - $b = r + c$ it's the **state width**, with $b = 25 \cdot 2^l$ for values $l = 0, 1, \dots, 6$
 - r is the **bit rate** (length of single blocks)
 - c is the **capacity** (security parameter)
 - SHA-3: Always $b = 1600$



The Keccak f -Permutation



- A **permutation** over b bits
- Variable number of rounds $r = 12 + 2l$
 - SHA-3: $l = 6$ and thus $r = 24$
- The functions θ, ρ, π, ι use XOR, AND, and NOT

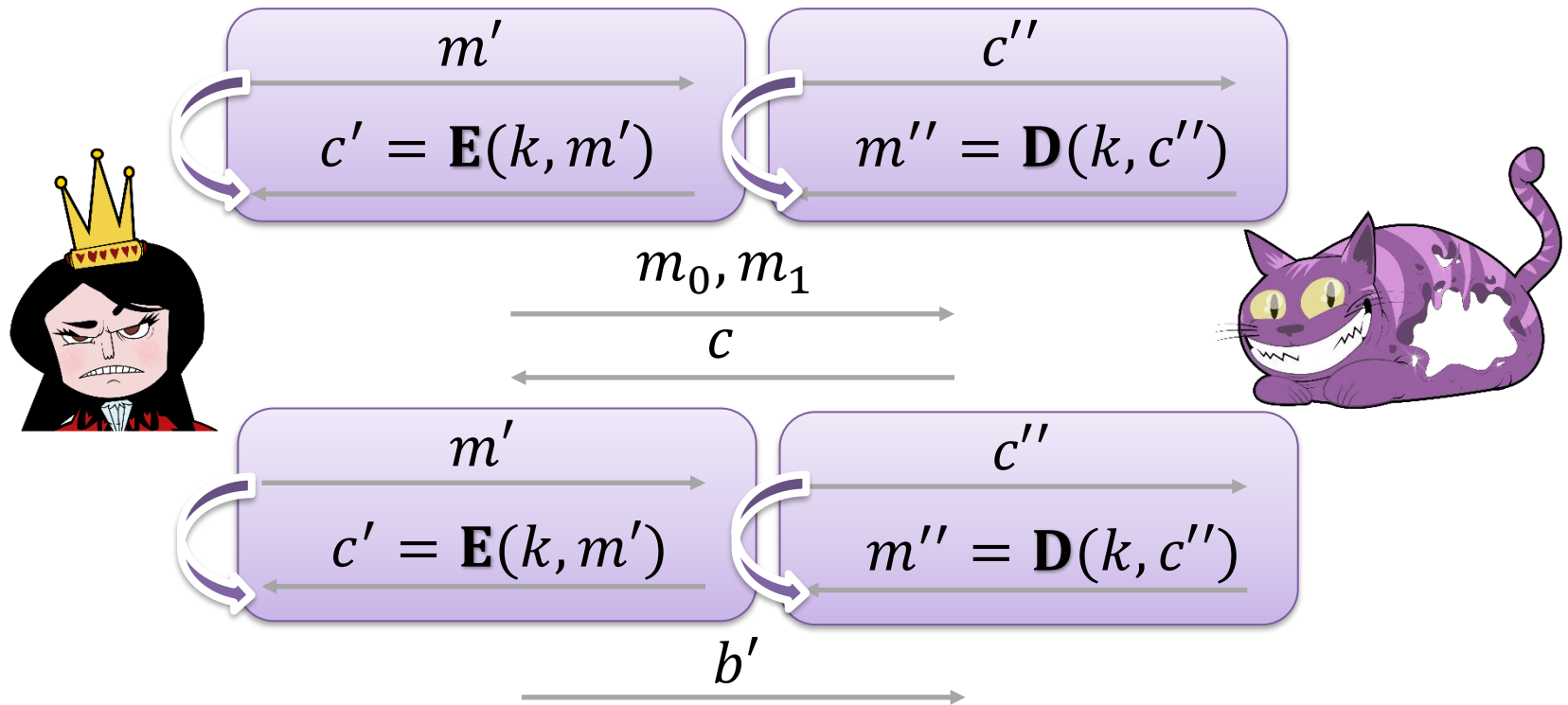


Combining Encryption and Authentication

- Encryption and authentication **separate goals**
- Can we achieve **both** at the same time?
- Intuitively we want that both
 - The ciphertext should hide the plaintext
 - It should be hard to compute a ciphertext without knowing the secret key
- This is called **authenticated encryption**



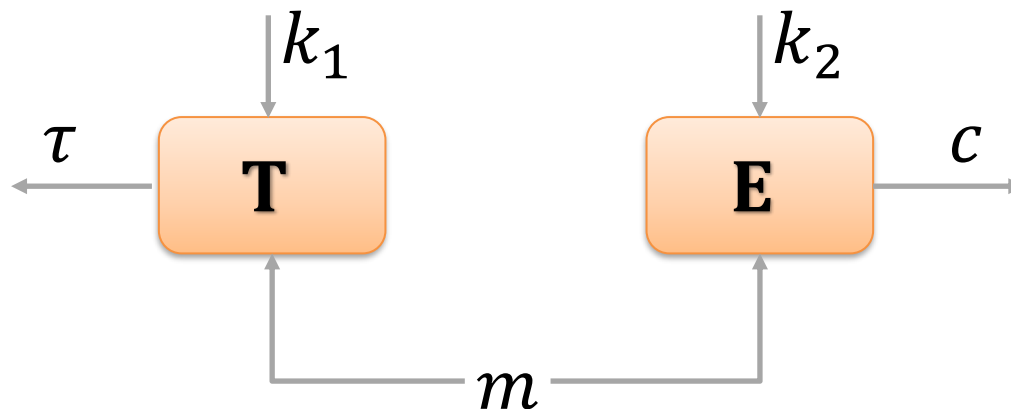
Chosen-Ciphertext Attacks (CCA) Security



- Both **encryption** and **decryption** queries
 - Cannot query on challenge ciphertext
- Captures **non-malleability**



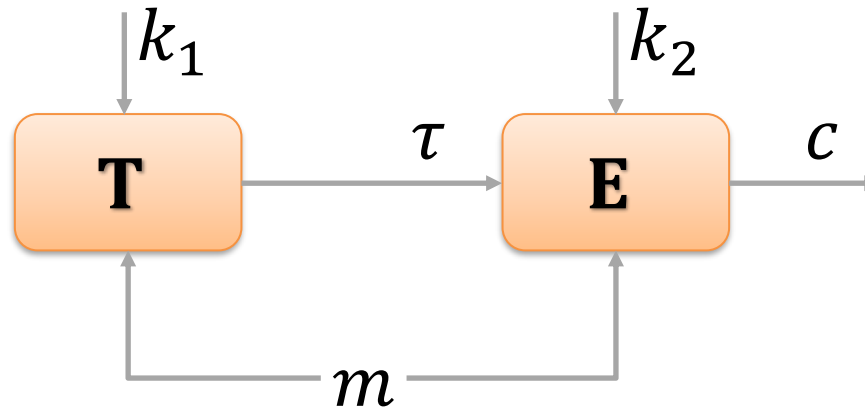
Encrypt-and-Authenticate



- Output: $c' = (c, \tau)$
- Insecure **in general**
 - Consider the function **T** that **reveals** the first bit of m ; this is **still** UF-CMA, but now c' is **not** even CPA secure



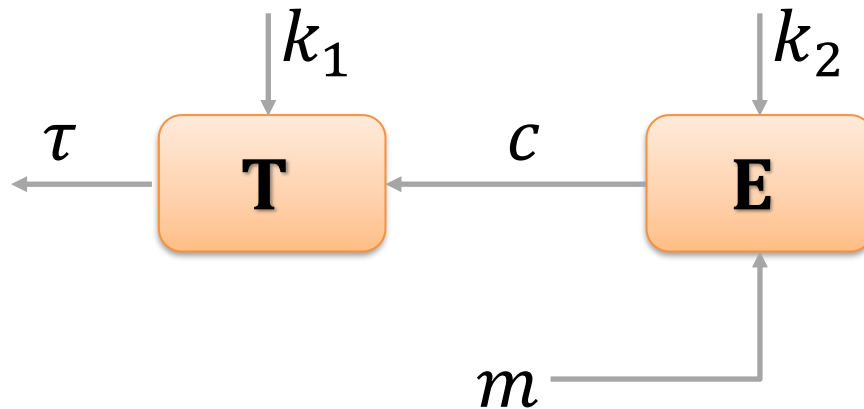
Authenticate-then-Encrypt



- Output: c
- Insecure **in general**
 - Consider the function **E'** that first encrypts m using a CPA secure **E** and then **encodes** each bit using two bits: $0 \rightarrow 00$ and $1 \rightarrow 01$ or 10
 - Ciphertexts containing 11 are **invalid**



Encrypt-then-Authenticate



- Output: $c' = (c, \tau)$
- **Always secure!**
 - For **any instantiation** of secure **E** and **T**

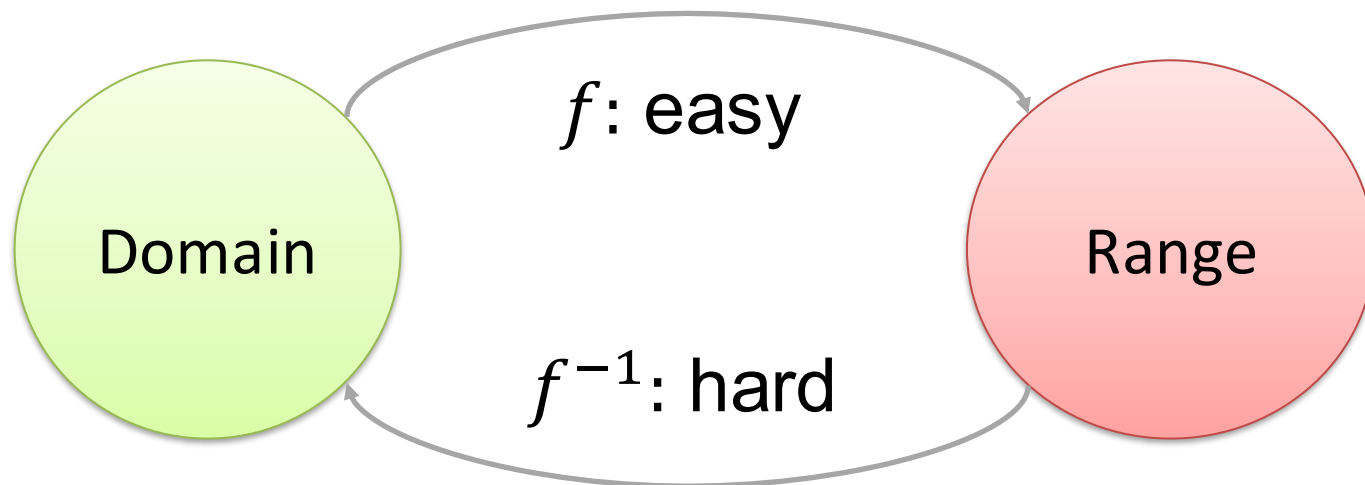


A Brief Tour of Minicrypt



One-Way Functions

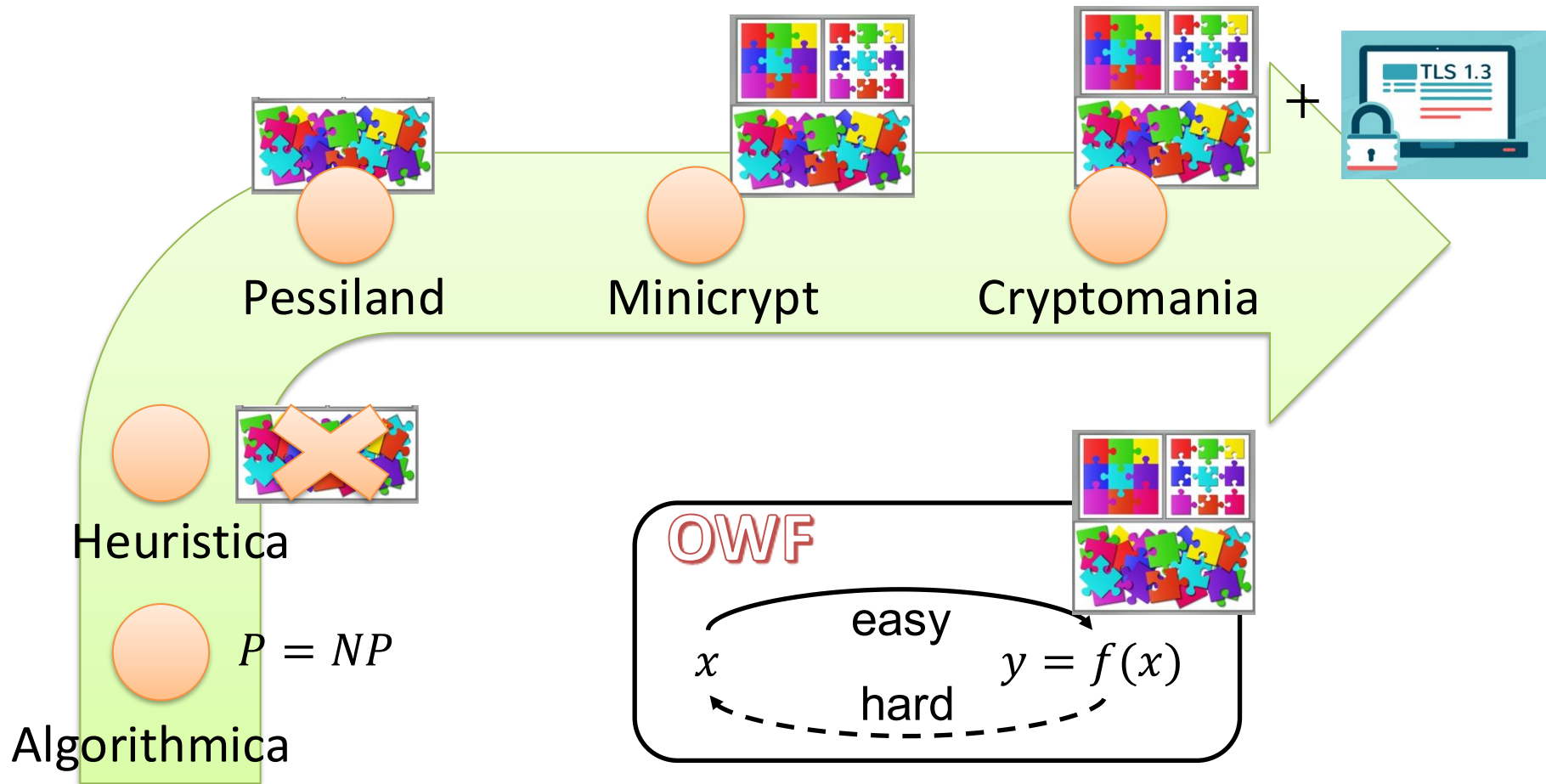
- Functions that are **easy to compute** but **hard to invert**



- Intimately connected to $P \neq NP$
- Minicrypt: There are OWFs but **no public-key cryptography** is possible

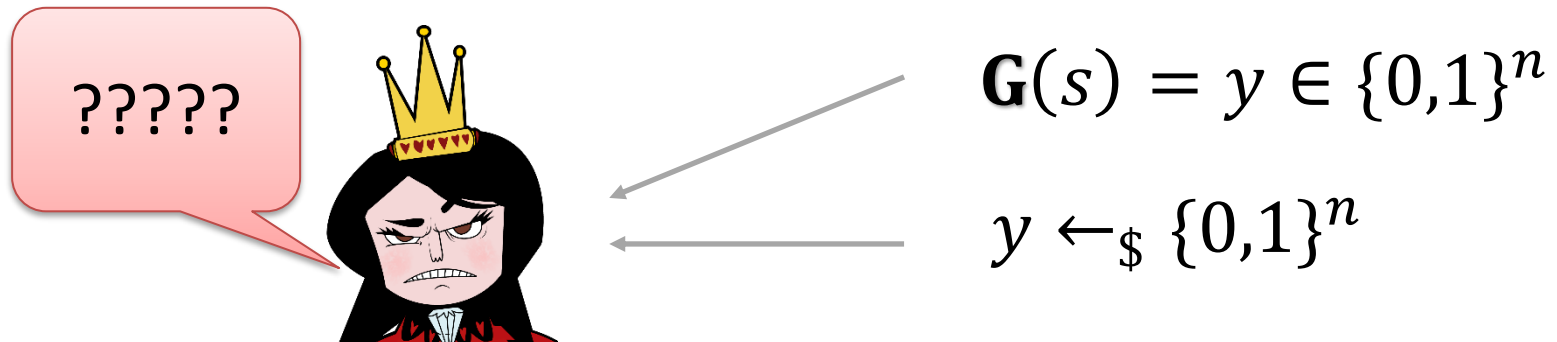


Impagliazzo's Worlds



Pseudorandom Generators

- A PRG expands a truly random (but **short**) seed into a **much longer** sequence that **looks random** (but it's not!)



- $\text{OWF} \Leftrightarrow \text{PRG} \Leftrightarrow \text{SKE}$ (one-time)
- One-time secure SKE: $\mathbf{E}(k, m) = \mathbf{G}(k) \oplus m$



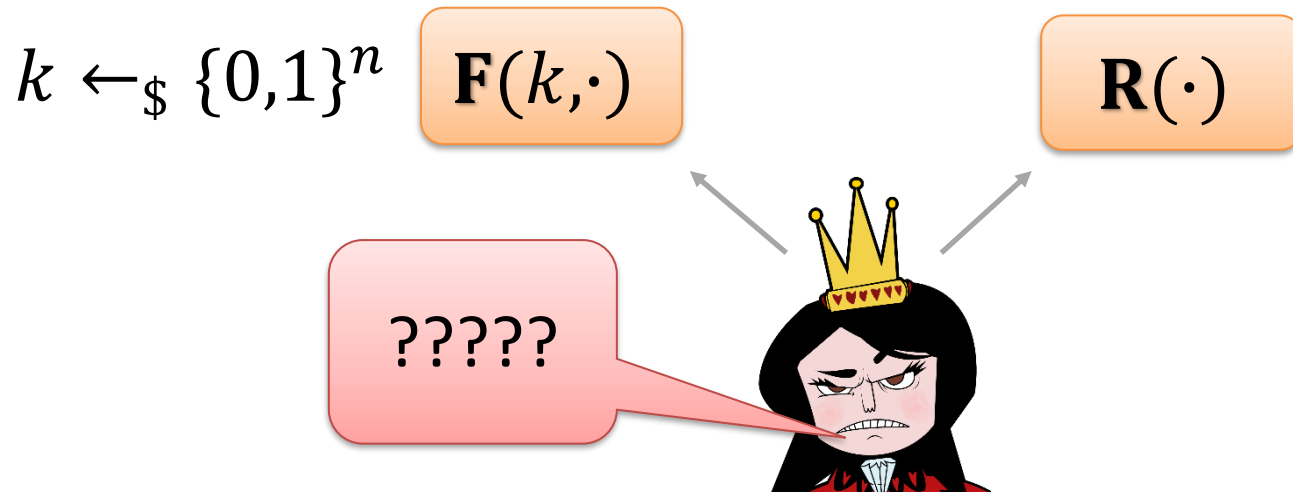
PRGs from OWFs

- Given y , which bits of x are **hard to compute**?
 - We know x is hard to compute, but maybe one can always compute **the first bit** of x
- **Hard-core bit**: We say h is hard core for f if given $y = f(x)$ it is hard to find the bit $h(x)$
 - **Fundamental fact**: Every OWF has a hard-core bit!
- If f is a one-way **permutation** (OWP), $\mathbf{G}(s) = f(s) || h(s)$ is a PRG with **1-bit stretch**
 - **Amplification**: Let $s_0 = s$, run $\mathbf{G}(s_i) = s_{i+1} || b_i$ for each $i = 0, 1, 2, \dots$ and output $b_1, b_2, \dots$



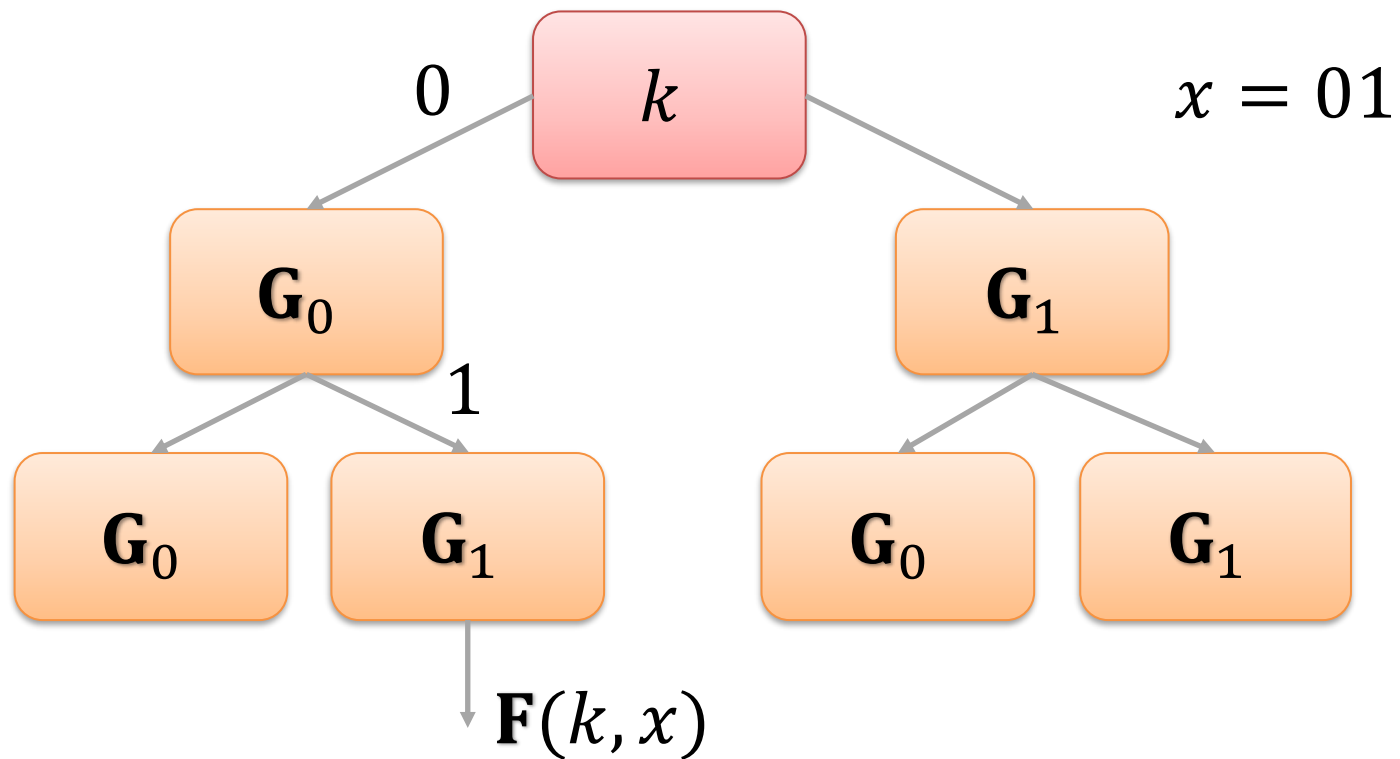
Pseudorandom Functions

- Consider a keyed function $\mathbf{F}(k, x)$ mapping $\{0,1\}^n$ into $\{0,1\}^n$ (for a fixed key k)
- Hard to distinguish $\mathbf{F}(k, \cdot)$ from **truly random function** $\mathbf{R}(\cdot)$

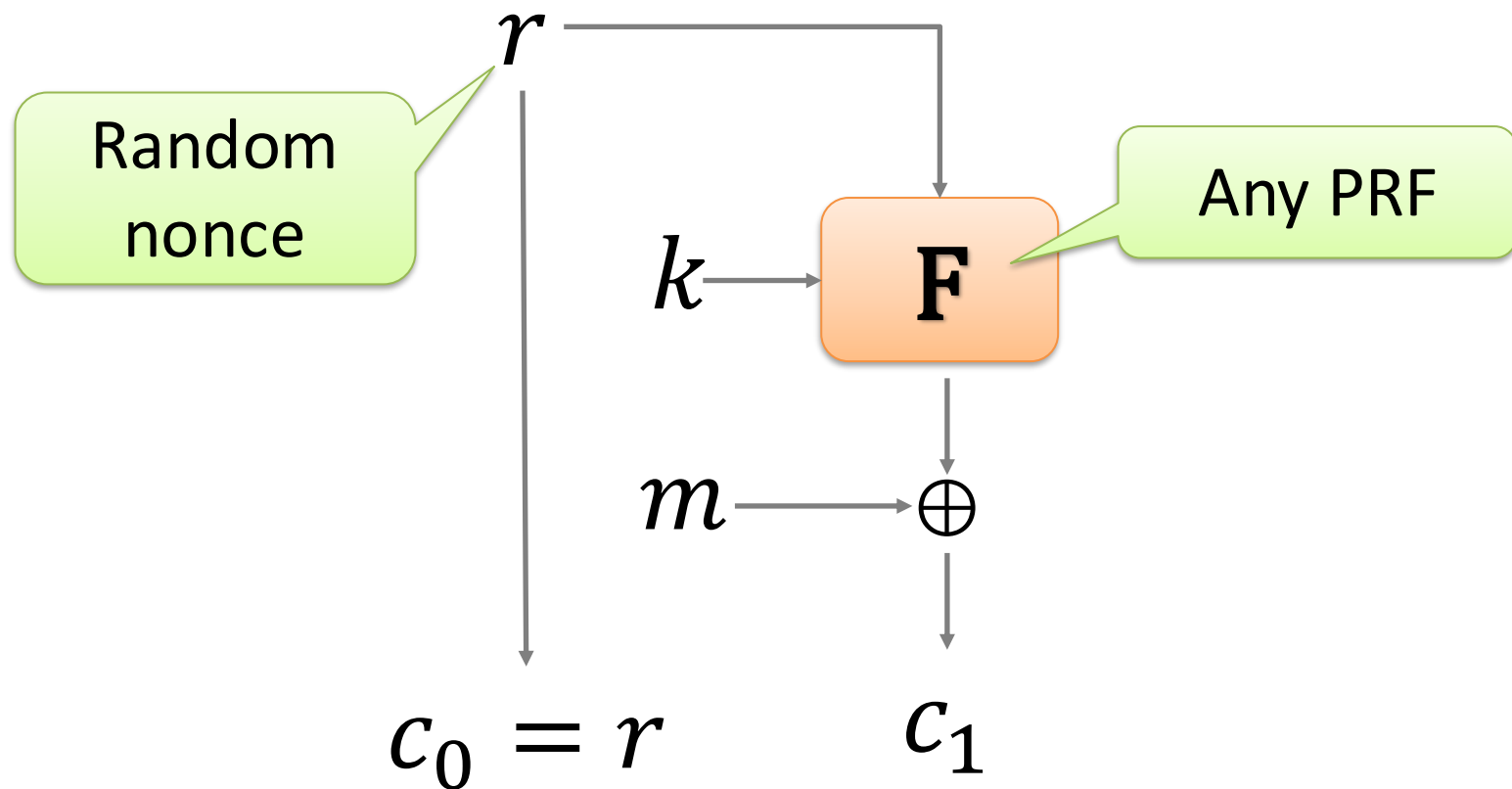


The GGM Tree

- $\text{PRG} \Rightarrow \text{PRF}$ (other direction also true)
- Let $\mathbf{G}(s) = (\mathbf{G}_0(s), \mathbf{G}_1(s))$ be a **length doubling** PRG



CPA-Secure SKE from PRFs

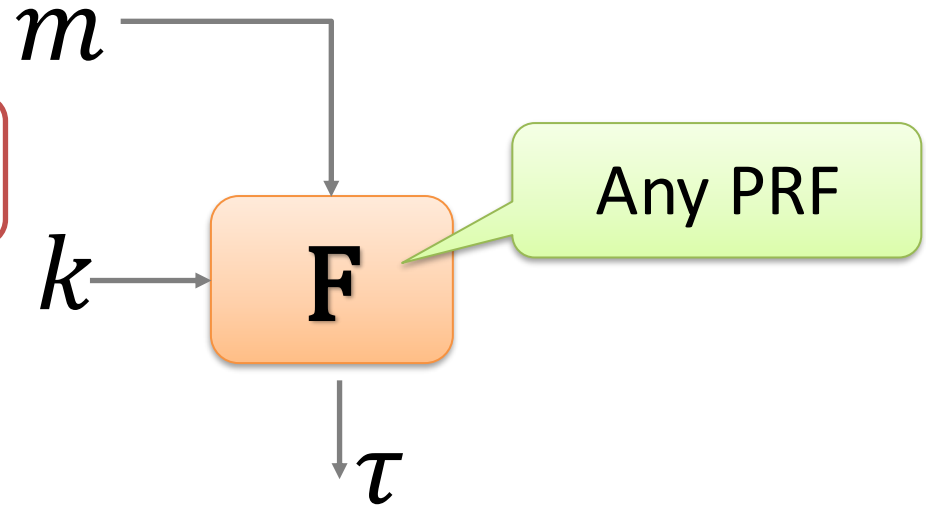


$$\mathbf{F}(k, c_0) \oplus c_1 = m$$



PRFs as MACs

$$\mathbf{T}(k, m) = \mathbf{F}(k, m)$$



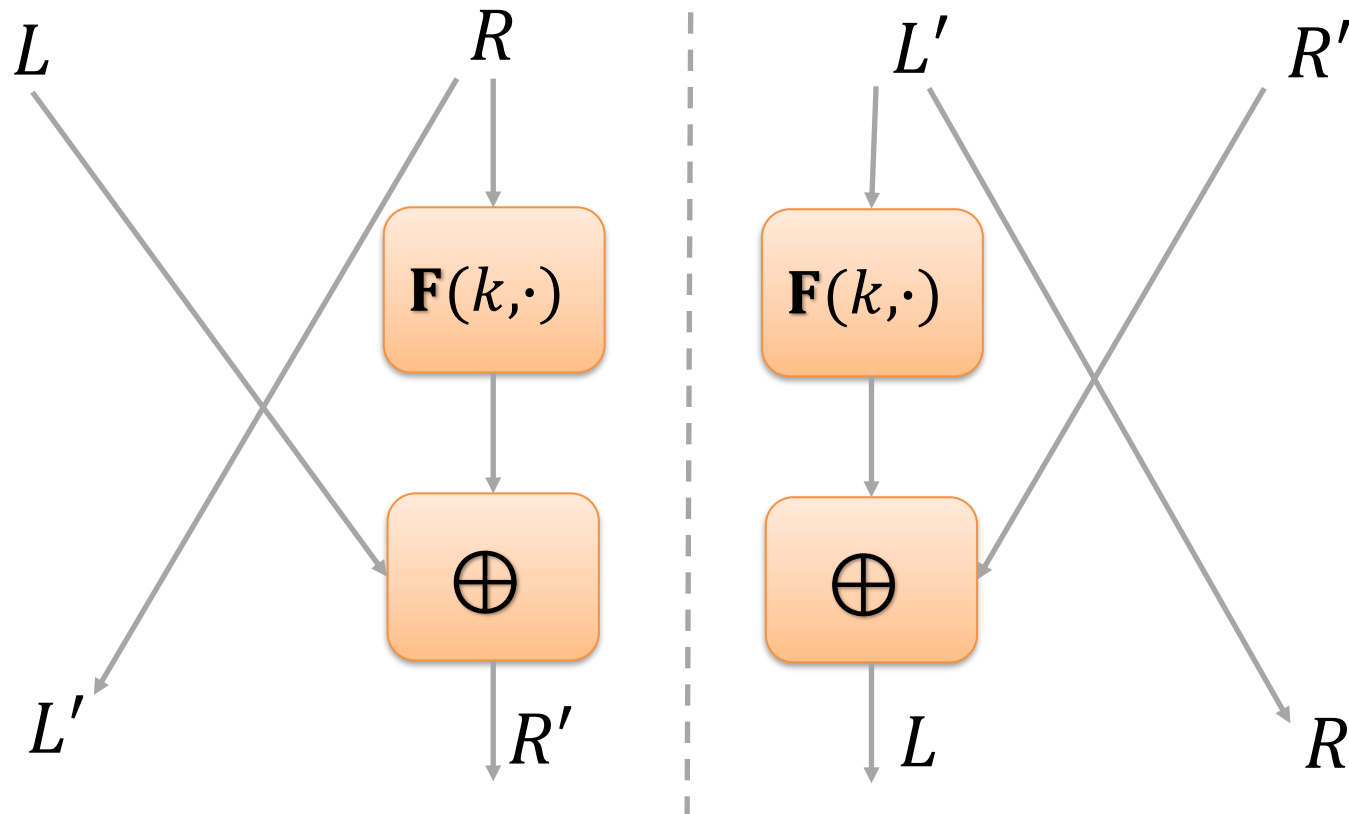
- Every PRF is a **fixed-length** MAC
- **Domain extension:**

CR Hash Function

$$\mathbf{T}'(k, m) = \mathbf{F}(k, \mathbf{H}(m))$$



Feistel Networks



$$\Psi_F(L, R) = (R, L \oplus \mathbf{F}(k, R))$$

$$\Psi_F^{-1}(L', R') = (R' \oplus \mathbf{F}(k, L'), L')$$



Luby-Rackoff Theorems

- Define the r -round **Feistel network** $\Psi_{\mathcal{F}}[r]$ as:

$$\Psi_{F_1, \dots, F_r}(L, R) = \Psi_{F_r} \left(\Psi_{F_{r-1}} \left(\dots \left(\Psi_{F_1}(L, R) \right) \right) \right)$$
$$\Psi_{F_1, \dots, F_r}^{-1}(L', R') = \Psi_{F_1}^{-1} \left(\Psi_{F_2}^{-1} \left(\dots \left(\Psi_{F_r}^{-1}(L', R') \right) \right) \right)$$

- Here, $\mathcal{F}$ is a family of PRFs (**independent** keys)
- Fundamental Fact: If $\mathcal{F}$ is a PRF, then $\Psi_{\mathcal{F}}[3]$ is a **pseudorandom permutation** (PRP)
 - And $\Psi_{\mathcal{F}}[4]$ is a **strong PRP** (i.e., adversary can access **inverse** permutation)



Public-key Cryptography

“Cubum autem in duos cubos, aut quadratoquadratum in duos quadratoquadratos, et generaliter nullam in infinitum ultra quadratum potestatem in duos eiusdem nominis fas est divider cuius rei demonstrationem mirabilem sane detexi. Hanc marginis exiguitas non caperet.”

Fermat's Last Theorem



Modular Arithmetic

- Quotient, remainder and gcd

$$a \bmod n = r \Rightarrow a = nq + r$$

– E.g., $2 \bmod 7 = 2$; $8 \bmod 7 = 1$

- **Congruences**: $a \equiv b \bmod n$ if n divides $a - b$
- $(\mathbb{Z}_n, +, \cdot)$ is a **ring**
 - If $\gcd(a, n) > 1$, then a **not invertible**
 - $\varphi(n) = \#\{a < n \text{ and co-prime with } n\}$
- $(\mathbb{Z}_p, +, \cdot)$ is a **field** (p is a prime)
 - $\mathbb{Z}_p^* = \{1, \dots, p - 1\}$; $\exists g$ a **generator**



Euclidean Algorithm

- **Lemma:** For all $a \geq b > 0$, $\gcd(a, b) = \gcd(b, a \bmod b)$
- **Theorem:** For all $a \geq b > 0$, we can find u, v such that $\gcd(a, b) = au + bv$
- Example: Take $a = 14$ and $b = 10$
 - $14 = 1 \cdot 10 + 4$; $10 = 2 \cdot 4 + 2$; $4 = 2 \cdot 2 + 0$ and in fact $\gcd(14, 10) = 2$

$$\begin{aligned} 2 &= 10 - 2 \cdot 4 = 10 - 2(14 - 1 \cdot 10) = -2 \cdot 14 + 3 \cdot 10 \\ &\Rightarrow (u, v) = (-2, 3) \end{aligned}$$



Basic Facts

- **Euler's Theorem**: Let $n > 0$. For all $a \in \mathbb{Z}_n^*$:
$$a^{\varphi(n)} \equiv 1 \pmod{n}$$
- **Corollary**: For a prime p and all a such that $p \nmid a$, we have $a^{p-1} \equiv 1 \pmod{p}$
- Example: Take $\mathbb{Z}_{10}^* = \{1, 3, 7, 9\}$
 - Note that $\varphi(10) = 4$
 - 3 is a generator: $3^0 \equiv 1, 3^1 \equiv 3, 3^2 \equiv 9, 3^3 \equiv 7$
 - For $a = 7$, we have $7^4 \equiv 1 \pmod{10}$



Primality Testing

- Every integer n is either a prime or it can be written as a product of primes (Euclid)
 - Such a prime decomposition is unique (Gauss)
- There are **infinitely many** primes (Euclid)
 - For large n there are $\approx \frac{n}{\ln(n)}$ primes in $[n]$ (PNT)
- We can **efficiently test** if an integer is a prime
 - Thanks to a famous algorithm by Agrawal, Kayal, and Saxena



Fermat's Test

- Given a value p to test, pick a random a not divisible by p and check if $a^{p-1} \equiv 1 \pmod{p}$
 - If not, conclude p is **composite**
 - If yes, conclude p is **probably prime**
- Let a be s.t. $a^{n-1} \equiv 1 \pmod{n}$ for composite n
 - a is a **Fermat liar**, and n is a **Fermat pseudoprime**
 - There are **infinitely** many Fermat pseudoprimes
 - There are **infinitely** many Carmichael numbers, i.e. numbers n for which **all** values of a co-prime with n are Fermat liars



Integer Factoring

- Let $n = p \cdot q$. Goal: Given n , find p, q
- Brute force: Divide n for all values $\leq \sqrt{n}$
 - Complexity $O(\log^2(n) \cdot p/\ln(p))$ is **exponential** in n whenever $p \approx \sqrt{n}$
- Many attempts and algorithms
 - Pollard, Quadratic and Number Field Sieve
 - Complexity is sub-exponential in n
- RSA challenges
 - Last challenge (RSA-768) took over 2 years on a **huge computer network**

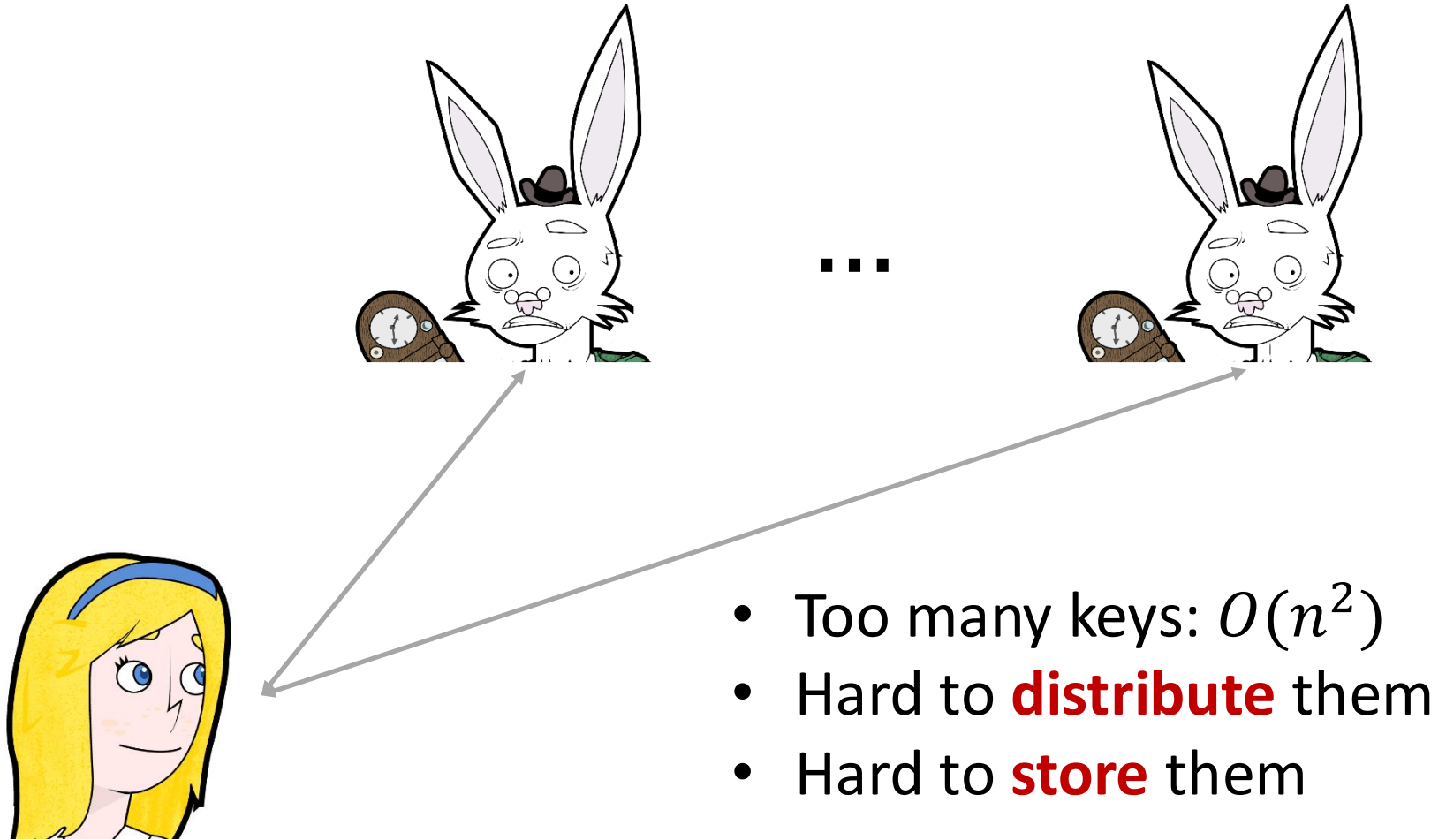


Discrete Logarithm

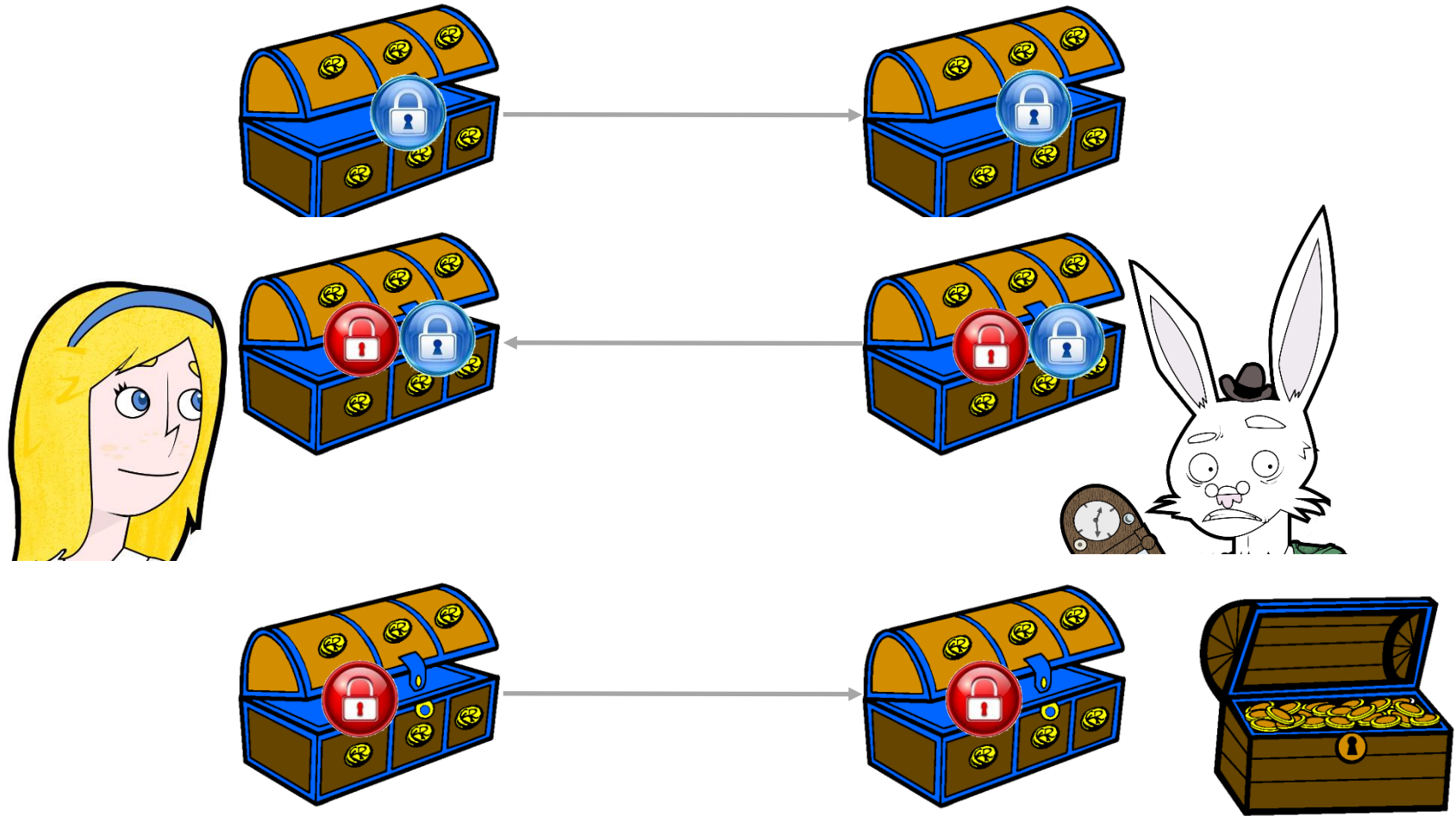
- Let $\mathbb{Z}_p^* = \{1, 2, \dots, p - 1\}$, for a prime p
- For each $y \in \mathbb{Z}_p^*$, $y = g^x$ for some x
- **Discrete Log assumption:** Given (y, g, p) compute x
 - I.e., modular exponentiation is a OWF
- Deeply studied problem
 - Best algorithms have complexity **sub-exponential** in the size of p



The Key Distribution Problem

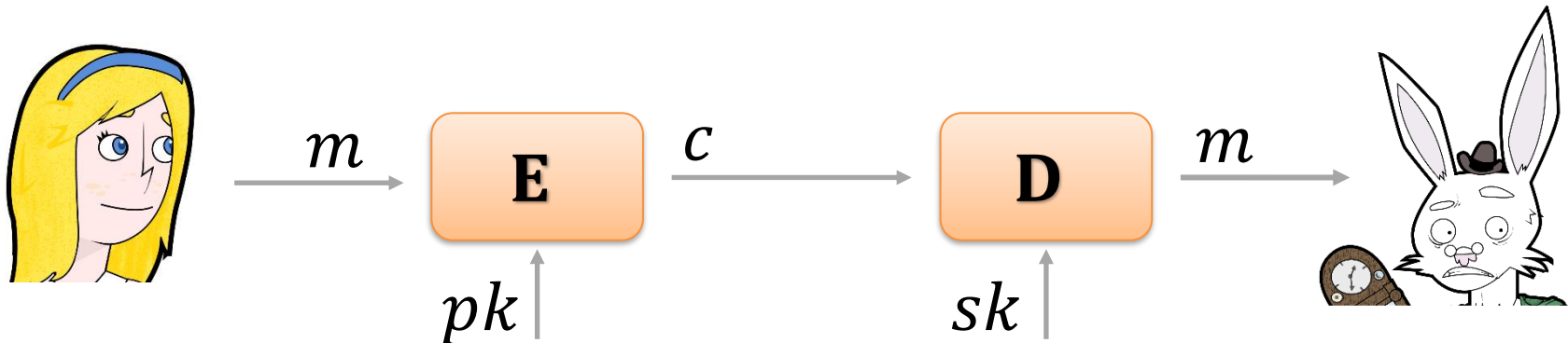


The Public-Key Revolution



Public-Key Encryption

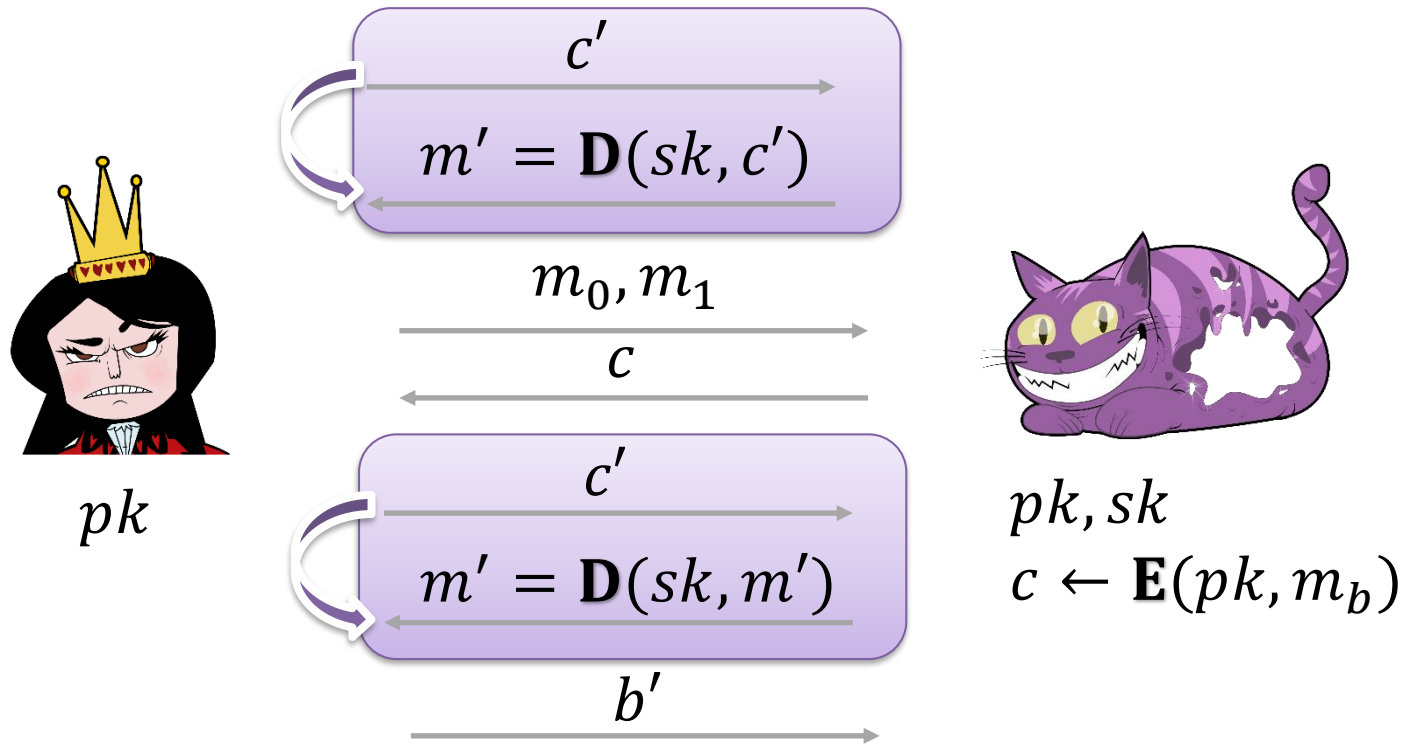
- Bob has a key pair (pk, sk)



- Must be **infeasible** to compute sk from pk
- No PKE scheme can achieve **unconditional security**



CPA/CCA Security



- No encryption queries (implicit given pk)
 - Cannot query on **challenge ciphertext**
- CCA security captures **non-malleability**



Hystory of PKE

- Concept **proposed** by Diffie & Hellman (1976)
- Rivest, Shamir, Adleman invent RSA (1978)
 - Very similar idea proposed by James Ellis in 1970 while working for GCHQ (but **top secret**)
- CPA security (Goldwasser and Micali – 1984)



Key Encapsulation

- PKE is one of the most significant advance in the 3000 years history of cryptography
- **Complementary** rather than a replacement of secret-key cryptography
 - PKE algorithms are **expensive**
 - Use PKE to share a secret (session) key and later encrypt communication with AES
 - Idea used in the Transport Layer Security (TLS) protocol (more on this later)



Textbook RSA

- Let $n = p \cdot q$ and e, d s.t. $e \cdot d \equiv 1 \pmod{\varphi(n)}$
- Set $pk = (n, e)$ and $sk = (d, p, q)$
- Encryption of $m \in \mathbb{Z}_n^*$: Return $c = m^e \pmod{n}$
- Decryption of $c \in \mathbb{Z}_n^*$: Return $c^d \pmod{n}$
 - **Correctness:** $c^d \equiv m^{ed} \equiv m$ (by Euler's Theorem)
- Parameters generation
 - Need to sample **large primes** (primality testing)
 - Sample e at random and compute the inverse of e modulo $\varphi(n)$ (using the **Euclidean algorithm**)



Remarks

- Need to **encode** bits in $\mathbb{Z}_n^*$
- Efficiency
 - Modular exponentiation: **"Square and multiply"**
 - Speed up using tricks from number theory
 - Small e makes encryption faster and $|pk|$ smaller
 - Harder to test for primality and need $|m| > |n|/3$
- Ciphertexts are **malleable**!
 - Given (m_1, c_1) and (m_2, c_2) , then $c_1 \cdot c_2$ is an encryption of $m_1 \cdot m_2$



RSA with Padding

- Clearly Textbook RSA is not CPA secure (why?)
- **Randomized** version
 - Encrypt $r||m$ (for random r)
 - Discard r on decryption
- PKCS#1.5 standard

$0||2||r \text{ (at least 8 bytes)}||0||m$

- First byte s.t. the obtained integer is $< n$
- Second byte encodes mode (i.e., encryption) and enforces **modular reduction**



The RSA Assumption

- Given $y = x^e$ (for $x \leftarrow_{\$} \mathbb{Z}_n^*$), compute x
- I.e., compute the **e -th root** modulo $n = p \cdot q$
- RSA **implies** Factoring
 - If one can factor n , it can also compute $\varphi(n)$ and thus d
- Other direction **not known**
 - But best algorithm for breaking RSA requires factoring the modulus
- Unfortunately, only little is known about **provable security** for RSA



RSA OAEP

- RSA with PKCS#1.5 padding is not **CCA-secure**
 - Celebrated attack by Bleichenbacher (more on this later)
- The PKCS#2.0 standard **overcomes** this limitation

$$L = m || 0 \dots 0$$

$$R = r \text{ (random)}$$

2-round Feistel with
random oracles G, H

$$\Psi_{G,H}(\cdot, \cdot)$$

$$c = (s || t)^e \bmod n$$

s

t



ElGamal PKE

- Let $h = g^x$ for g a **generator** of $\mathbb{Z}_p^*$ and random x
- Set $pk = (g, p, h)$ and $sk = x$
- Encryption of $m \in \mathbb{Z}_p^*$: Pick **random** r and return $c = (c_1, (g^r, h^r \cdot m))$
- Decryption of $(c_1, c_2) \in (\mathbb{Z}_p^*)^2$: Return c_2 / c_1^x
 - $\frac{c_2}{c_1^x} = \frac{h^r \cdot m}{(g^r)^x} = \frac{h^r \cdot m}{(g^x)^r} = \frac{h^r \cdot m}{h^r} = m$



The CDH Assumption

- Let $\mathbb{Z}_p^* = \{1, 2, \dots, p - 1\}$, for a prime p and let g be a **generator**
- Given (g, p, g^x, g^y) for random x, y , **compute** g^{xy}
- CDH **implies** DL
 - If we can solve DL, we can compute x (or y) and thus obtain $g^{xy} = (g^y)^x$ (or $g^{xy} = (g^x)^y$)
- Other direction **not known**
 - But best algorithm for solving CDH requires to compute a DL



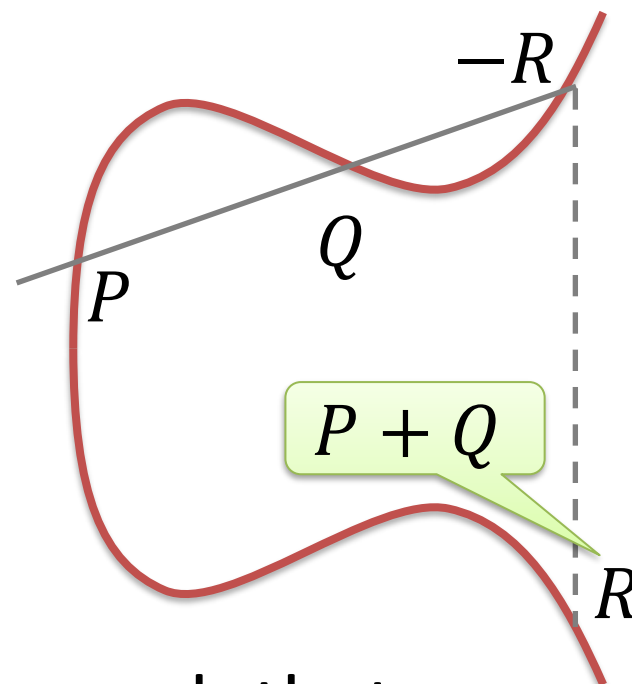
The DDH Assumption

- **Distinguish** (g, p, g^x, g^y, g^z) for random x, y, z from (g, p, g^x, g^y, g^{xy})
- DDH **implies** CDH
 - If we can solve CDH, we can compute g^{xy} and thus distinguish between g^{xy} and g^z
- Other direction is **false**
 - Simply because DDH **does not hold** in $\mathbb{Z}_p^*$
 - Take $p = 2q + 1$ (for primes p, q) and let $\mathbb{G}$ be the subgroup of $\mathbb{Z}_p^*$ consisting of all elements $y = x^2$
 - The order of $\mathbb{G}$ is q



Elliptic-Curve Cryptography

- The points of a curve $E: Y^2 = X^3 + aX + b$ modulo a prime p form a **group** $\mathbb{G} = (E, +)$
- Discrete logarithm: Given $Q = xP$, find x
 - DDH believed to be hard



- **Bilinear map** $e: \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$ such that

$$\forall a, b: e(g^a, g^b) = e(g, g)^{ab}$$



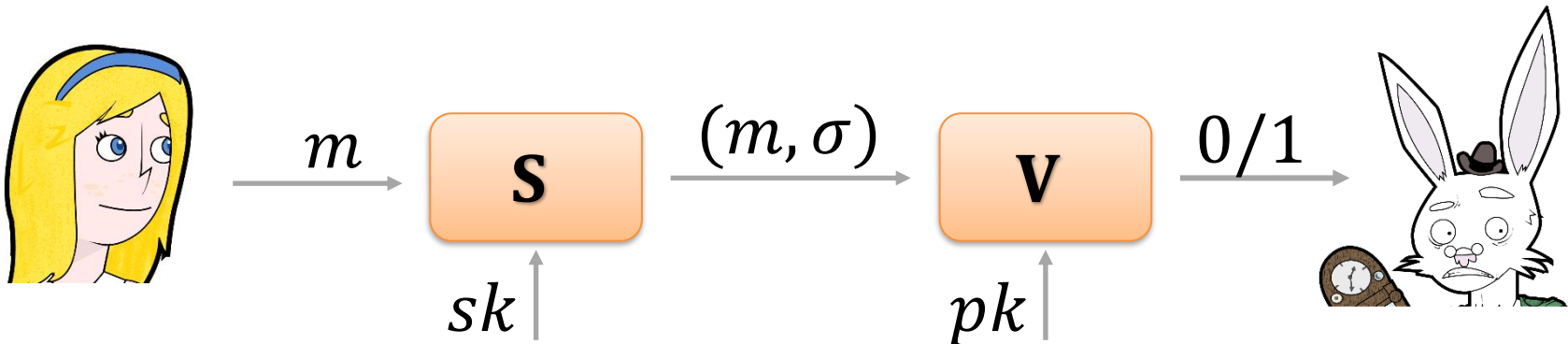
The Bilinear Diffie-Hellman Assumption

- When $\mathbb{G}$ is a **bilinear group**, DDH is easy in $\mathbb{G}$
 - Given (g, p, e, X, Y, Z) can simply check that $e(X, Y) = e(g, Z)$
- However, the following **variant of CDH** is believed to hold:
 - Given (g, p, e, g^x, g^y, g^z) compute $e(g, g)^{xyz}$
 - Once again BCDH **implies** CDH, but the other direction is **unknown**
- One can also assume DDH to be **hard** in $\mathbb{G}_T$
 - This is called the **SXDH assumption**



Digital Signatures

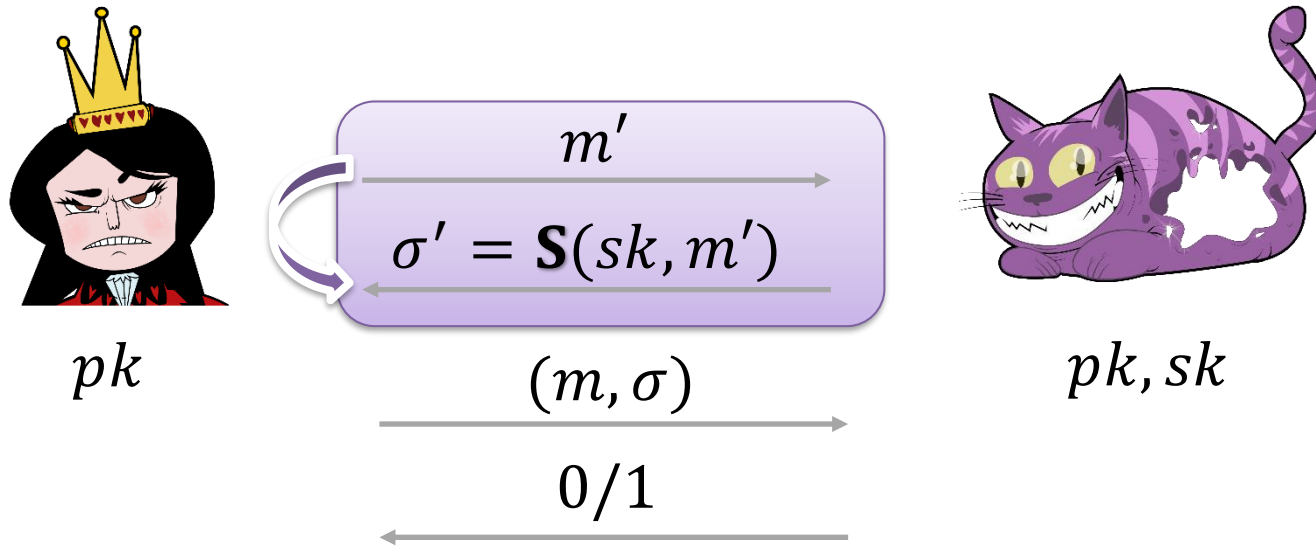
- Alice has a key pair (pk, sk)



- Correctness:** $\forall pk, sk, m: \mathbf{V}(pk, (m, \mathbf{S}(sk, m))) = 1$
- Security:** Should be hard to forge a signature on a message without knowing the secret key



Unforgeability



- Adversary wins iff (m, σ) is **valid** and m is **fresh** (i.e. not asked during signing queries)



How to Sign with RSA

- Let $n = p \cdot q$ and e, d s.t. $e \cdot d \equiv 1 \pmod{\varphi(n)}$
- Set $pk = (n, e)$ and $sk = (d, p, q)$
- Signature of $m \in \mathbb{Z}_n^*$: Return $\sigma = m^d \pmod{n}$
- Verification of $(m, \sigma) \in \mathbb{Z}_n^* \times \mathbb{Z}_n^*$: Return 1 iff $\sigma^e = m \pmod{n}$
 - **Correctness**: $\sigma^e \equiv m^{ed} \equiv m$ (by **Euler's Theorem**)
- **Not secure!**
 - Pick any $\sigma \in \mathbb{Z}_n^*$ and output (m, σ) where $m = \sigma^e \pmod{n}$



Full-Domain Hash

- Solution: First **hash** the message!
- Now, $\sigma = (\mathbf{H}(m))^d$ where $\mathbf{H}$ is a cryptographic hash function
- Possible attacks:
 - Pick any σ and let $\mathbf{H}(m) = \sigma^e$; hard to compute m from $\mathbf{H}(m)$ (**one-wayness**)
 - Given valid (m, σ) , hard to find $m' \neq m$ s.t. $\mathbf{H}(m') = \mathbf{H}(m)$ (**weak collision resistance**)
 - Find $m \neq m'$ s.t. $\mathbf{H}(m) = \mathbf{H}(m')$ and obtain a valid signature on m (**strong collision resistance**)



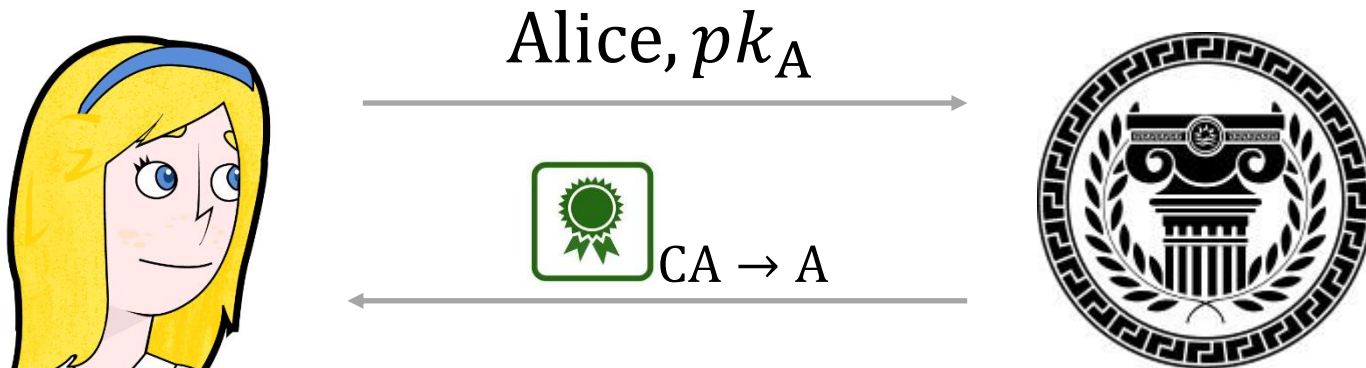
Public-Key Infrastructure

- Need to **certify** public keys
 - Otherwise simple **man-in-the-middle** attacks are possible
- Have a Certification Authority (CA) confirm the **authenticity** of public keys
 - CA signs binding between identity and public key
- Single CA not a good solution
 - Unique point of failure
 - In practice: **Chains** of certificates



Basic Idea

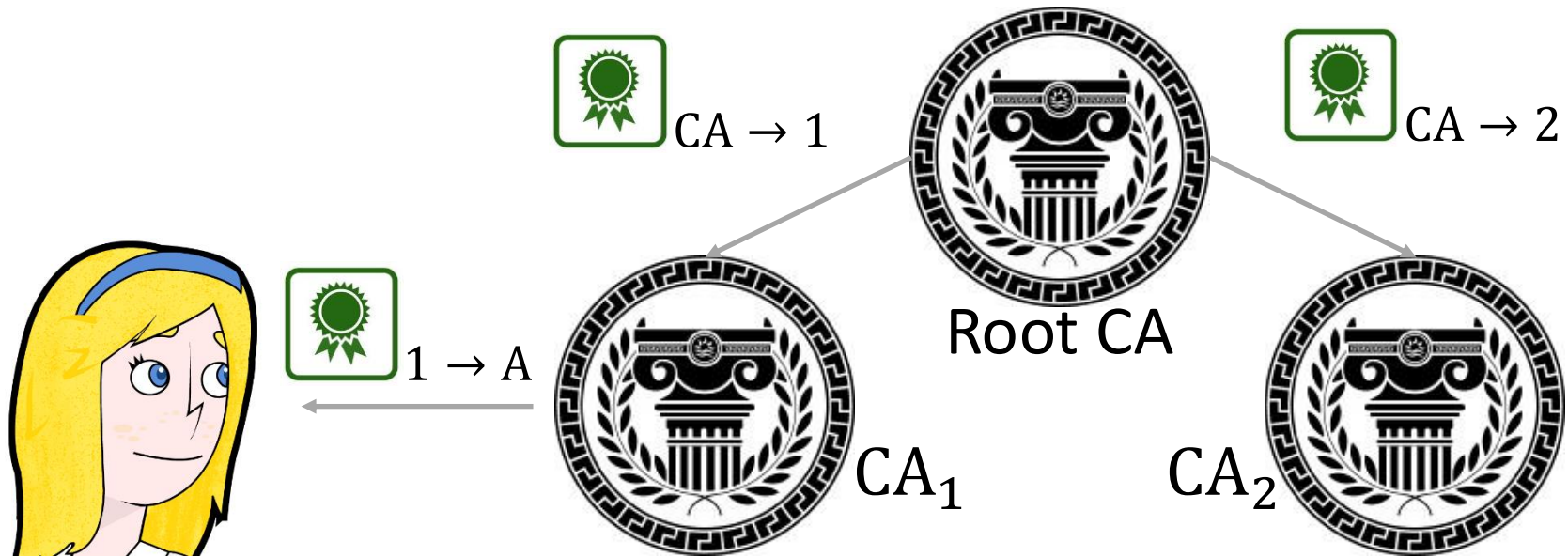
$$\boxed{\text{Sun}}_{CA \rightarrow A} \leftarrow \mathbf{S}(sk_{CA}, pk_A || \text{Alice})$$



- Format of certificates **standardized** by ITU (X.509)
- Anybody can verify the certificate using pk_{CA}



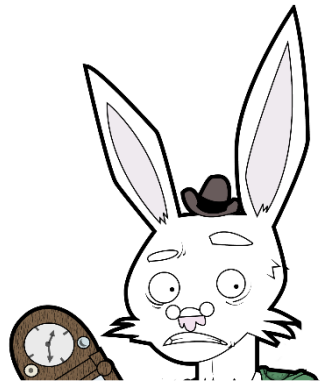
Chain of Certificates



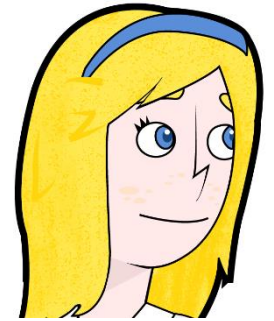
- Propagation of trust
- Each user might be a member of **different** PKIs
- Typically the binding involves credentials
 - User name only for management and auditing



Web of Trust



$pk_B, \boxed{\text{Sun}}_{E \rightarrow B} \boxed{\text{Sun}}_{F \rightarrow B} \boxed{\text{Sun}}_{G \rightarrow B}$



pk_D, pk_E, pk_F

- Users can **self-certify** public keys
 - No trusted party required (fully distributed environment)
 - Approach taken in PGP (IETF)



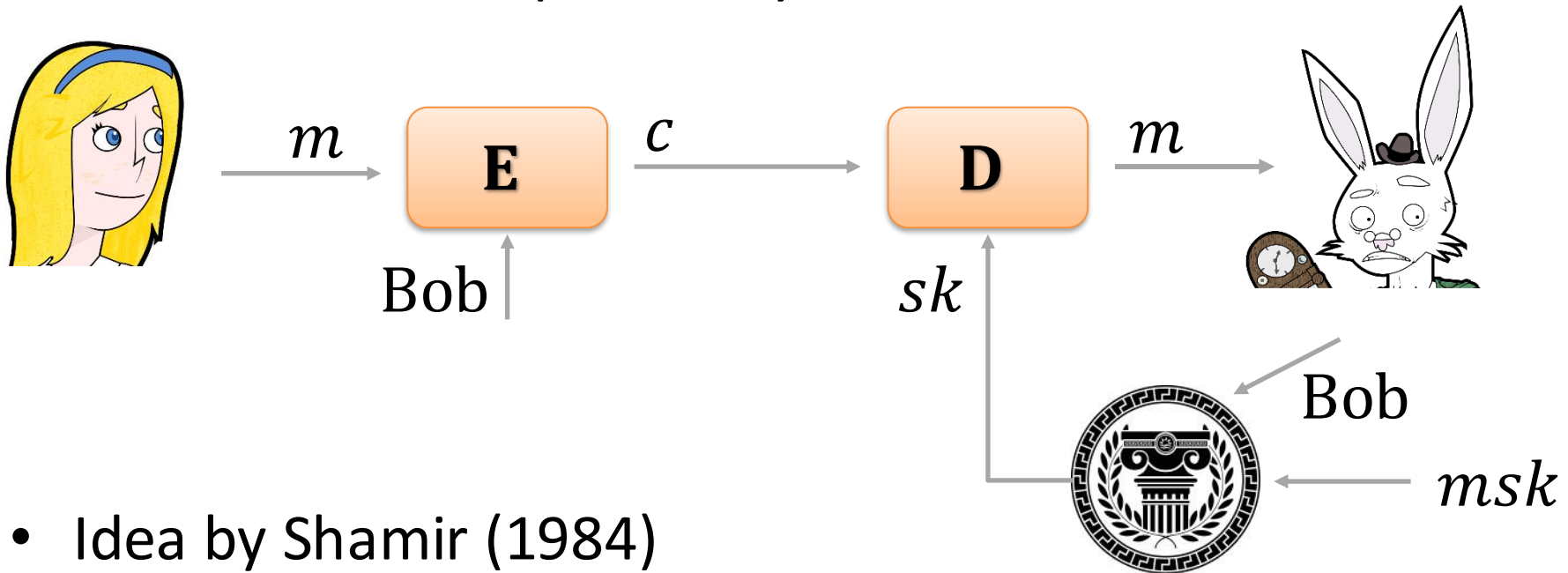
Management of Certificates

- Certificates should have associated a **validity interval**
 - Need to check it didn't expire
 - What granularity?
- **Revocation** of certificates
 - Associate a different serial number to each certificate
 - Maintain a Certificate Revocation List
- CA must be **online** in order to answer validation queries



Identity-Based Encryption (IBE)

- Use **identities** as public keys



- Idea by Shamir (1984)
 - First realization by Boneh and Franklin (2001)

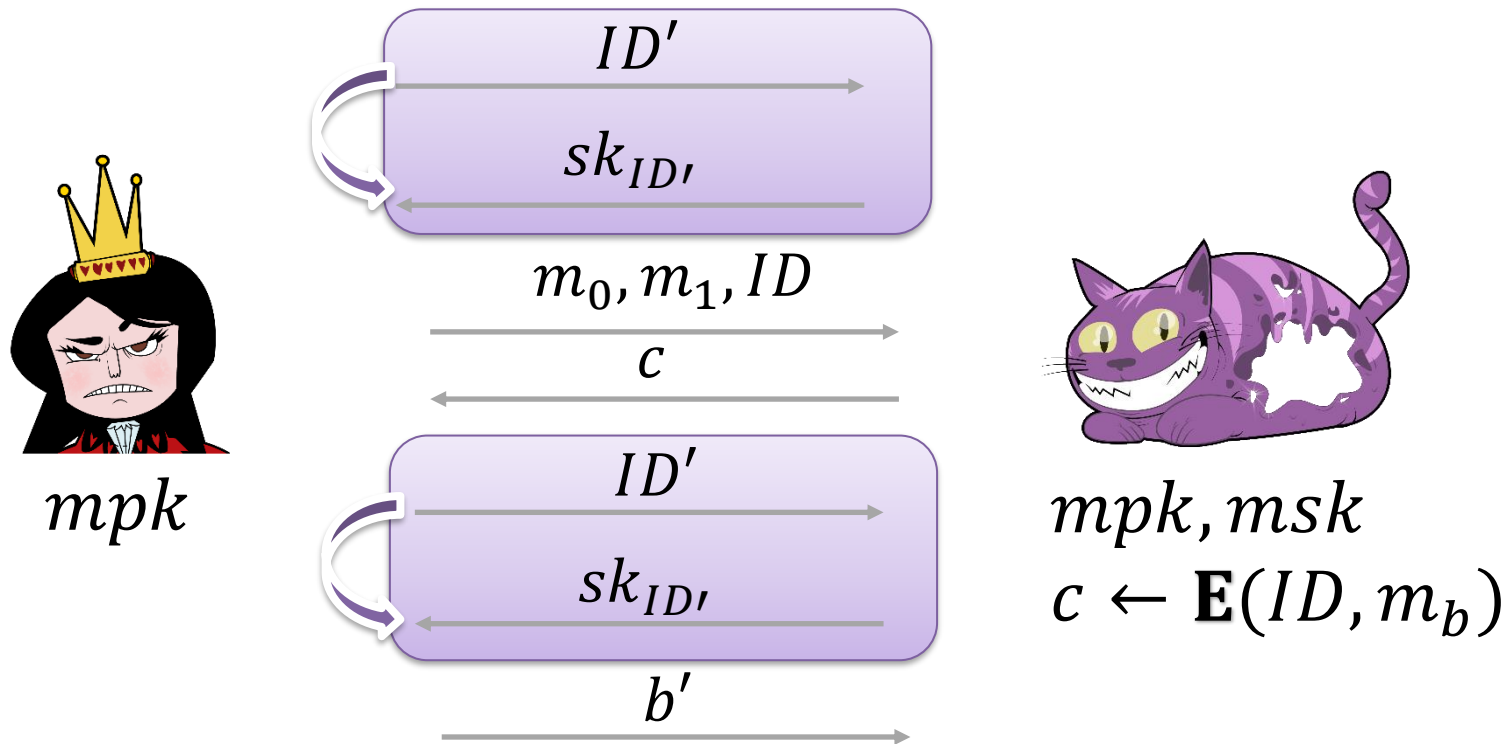


IBE vs. PKI

- Keys are **unrevocable** in IBE
 - Generate keys w.r.t. identity plus some timestamp
- PKI **not necessary** in IBE
- KGC is a high-value target to adversaries
 - Several mitigations are possible
- **Key escrow** (not present in PKI)
- IBE requires a **secure channel** in order to transmit the secret key to the user



Semantic Security of IBE



- Adversary **not allowed** to extract the secret key for the **challenge identity** ID

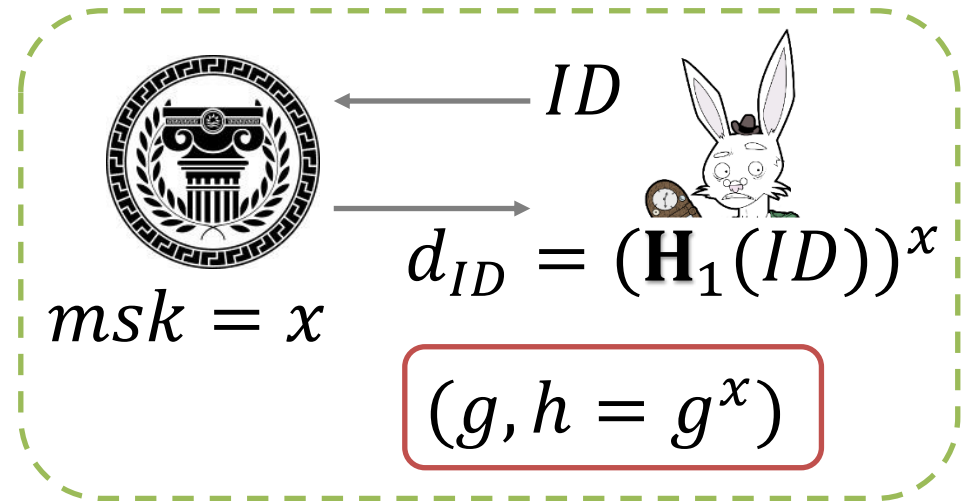


Boneh-Franklin IBE

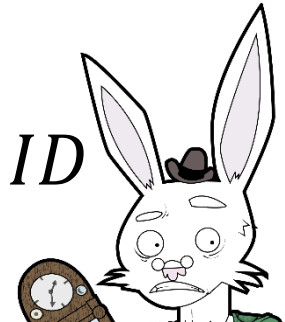


Pick
random r

$$y_{ID} = e(\mathbf{H}_1(ID), h)$$



$$c = (u, v) = (g^r, m \oplus \mathbf{H}_2(y_{ID}^r))$$



ID

$$\begin{aligned} e(d_{ID}, u) &= e((\mathbf{H}_1(ID))^x, g^r) \\ &= e((\mathbf{H}_1(ID), g)^{xr} = y_{ID}^r \end{aligned}$$

$$m = v \oplus \mathbf{H}_2(e(d_{ID}, u))$$

